

Tema 1

Sistemas de numeración

Convenio o criterio utilizado para representar las cantidades numéricas.

Todo sistema de Numeración se caracteriza por su Base.

Base 10 $\Omega = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$

Base 2 o binario $\Omega = \{0, 1\}$ = Binary digits

Hexadecimal = base 16 $\Omega = \{0, 1-9, A, B, C, D, E, F\}$
10, 11, 12, 13, 14, 15

Conversion entre bases

En cualquier sistema de Numeración una cantidad "N" se puede representar

como: $N/b = PE, PF/b$

$\uparrow \quad \uparrow$
Parte Entera Parte Fraccionaria

Teorema Fundamental de los Sistemas de Numeración

Conversión a base 10 de una cantidad $N(b)$

$$N/b \rightarrow N/10 = \dots + n_2 b^2 + n_1 b^1 + n_0 b^0 + n_{-1} b^{-1} + n_{-2} b^{-2} \dots = \sum_i n_i b^i$$

$\xleftarrow{\text{PE}}$
 $\xrightarrow{\text{PF}}$

n_i : dígito de orden "i"
 b^i : peso del dígito de orden

Ejemplo:

$111011011'011(2)$
 $PE \leftarrow L \rightarrow PF$
 $1 \cdot 2^8 + 1 \cdot 2^7 + 1 \cdot 2^6 + 1 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^1 + 1 \cdot 2^0 + 1 \cdot 2^{-2} + 1 \cdot 2^{-3} =$
 $= 256 + 128 + 64 + 16 + 8 + 2 + 1 + 0,25 + 0,125 =$
 $= 475,375(10)$

$$N(10) \rightarrow N(6)$$

Se procederá realizando divisiones enteras o "módulo b " de forma sucesiva sobre PE hasta que el cociente sea inferior a " b " y multiplicaciones enteras o "módulo b " de forma sucesiva sobre PF hasta que sea 0.

Ejemplo:

$$475,375(10) \rightarrow N(16)$$

$$\begin{array}{r} 475 \overline{) 16} \\ 155 \ 29 \ 16 \\ \hline 11 \ 13 \ 1 \end{array}$$

$$PF = 475(10) = 1DB(16)$$

$$\begin{array}{r} 0,375 \\ \times 16 \\ \hline 6 \ 0 \ 0 \end{array}$$

$$PF = 0,375(10) = 0,6(16)$$

$$475,375(10) = 1DB,6(16)$$

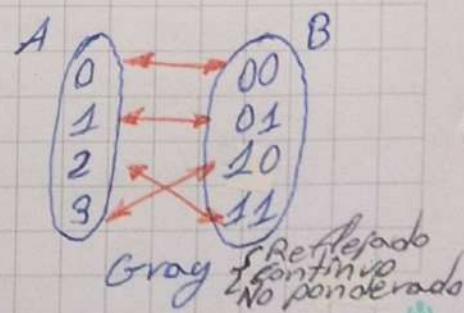
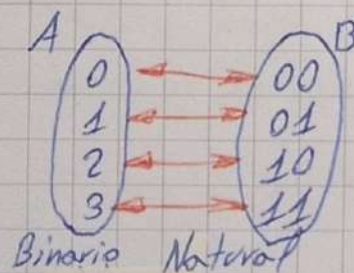
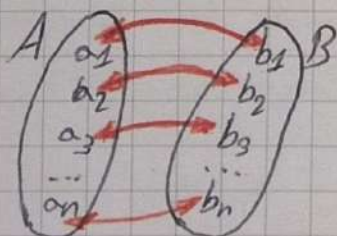
Se puede pasar de una base a otra sin pasar por base 10 siempre que exista la relación: $a^n = b$.

Se puede pasar de base $2 \rightarrow 8$ y viceversa pues $2^n = 8$ $n=3$ indicando que una cifra en base 8 corresponde a tres cifras en base 2.

Octal	Binario
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

Códigos

Aplicación biunívoca, 1 a 1, entre los elementos de dos conjuntos, A y B, de manera que podremos representar los elementos de un conjunto por los elementos que les corresponden en el otro conjunto.



El conjunto A será un subconjunto de los números naturales, $A \subseteq N = \{0, \infty\}$
y el conjunto B será un subconjunto de las combinaciones binarias
de "n" bits $\rightarrow 2^n!$ posibles códigos

Codifica los números naturales en base 2.

1 bit $\rightarrow 2^1$ combinaciones

3 bit $\rightarrow 2^3$ combinaciones

4 bit $\rightarrow 2^4$ combinaciones

n bits $\rightarrow 2^n$ combinaciones

Los códigos que hacen uso de todas las combinaciones que se pueden obtener con n bits con que se forman reciben el nombre de códigos completos.

- Dos combinaciones binarias son "adyacentes" si se diferencian en uno solo de sus bits.

10001 ~~X~~ 11101
No

- Un código es "continuo" cuando cada combinación del código es adyacente con la que le precede y con la que le sucede y, además, la última y primera combinación también son adyacentes.

Se forma el código Gray de $(n-1)$ bits y se refleja, las combinaciones de la mitad superior se completan con 0 y la mitad inferior con 1.

Gray de 1 bit

Gray 2 Bit

0	0
0	1
1	1
1	0

Red arrow pointing to the right.

Códigos BCD (Binary Coded Decimal)

- Se utilizan para codificar los 10 dígitos decimales, $\{0-9\}$
- Por tanto, todos los códigos BCD están formados por 10 combinaciones "al menos 4 bits", codificándose, en una cantidad cada dígito de forma individual.

	BCD Natural Binario Natural	BCD Aiken*	BCD Exceso 3 Empieza a codificar el 0 desde el 3 binario.
(0)	0 0 0 0	0 0 0 0	0 0 1 1
(1)	0 0 0 1	0 0 0 1	0 1 0 0
(2)	0 0 1 0	0 0 1 0	0 1 0 1
(3)	0 0 1 1	0 0 1 1	0 1 1 0
(4)	0 1 0 0	0 1 0 0	0 1 1 1
(5)	0 1 0 1	1 0 1 1	1 0 0 0
(6)	0 1 1 0	1 1 0 0	1 0 0 1
(7)	0 1 1 1	1 1 0 1	1 0 1 0
(8)	1 0 0 0	1 1 1 0	1 0 1 1
(9)	1 0 0 1	1 1 1 1	1 1 0 0

* Código reflejado y complementario: se reflejan las 5 combinaciones superiores sobre la mitad inferior y se complementan a 1 (se reemplazan los 0 por 1 y los 1 por 0).

Códigos ponderados

Un código es ponderado cuando podemos asignar a cada bit del código un valor numérico, que llamaremos peso, de forma que para obtener el equivalente decimal de una combinación del código habrá que sumar los pesos de los bits que se encuentran a "1" en dicha combinación.

El binario natural es ponderado, al obtener el sistema de ecuaciones este admite solución.

(2) (1) ~~+~~ Pesos

P_1

P_0

0 0 (0)

0 1 (1)

$$P_0 = 1$$

1 0 (2)

$$P_1 = 2$$

1 1 (3)

$$P_1 + P_0 = 3$$



Tema 2 Lógica Binaria

Definición variable lógica o binaria

- Es aquella que puede tomar los dos posibles valores binarios (0, 1).
- Las utilizaremos para codificar ocurrencias o propiedades del mundo real por medio de combinaciones binarias de forma que dicha información pueda ser manejada (procesada) por un circuito digital.

Algebra de Boole

Un conjunto $B = \{a, b, c, \dots\}$ de variables lógicas o binarias, dotado de los operadores suma lógica, producto lógico e inversión, $(+, \cdot, -)$, es un algebra de Boole si y solo si verifica los cuatro axiomas:

Axioma 1 Los operadores lógicos "+" y "·" son conmutativos.

$$\forall a, b \in B \Rightarrow \begin{cases} a + b = b + a \\ a \cdot b = b \cdot a \end{cases}$$

Axioma 2 Existen dos elementos neutros "0" y "1" que verifican:

$$\forall a \in B \Rightarrow \begin{cases} a + 0 = a \\ a \cdot 1 = a \end{cases}$$

Axioma 3 Se verifica la propiedad distributiva (= factor común).

$$\forall a, b, c \in B \Rightarrow \begin{cases} a + (b \cdot c) = (a + b) \cdot (a + c) \\ a \cdot (b + c) = (a \cdot b) + (a \cdot c) \end{cases}$$

Axioma 4 Ley del complementario

$$\forall a \in B \exists \bar{a} \in B / \begin{cases} a + \bar{a} = 1 \\ a \cdot \bar{a} = 0 \end{cases}$$

Teoremas derivados de los axiomas

Teorema de la idempotencia: Propiedad para realizar una acción varias veces y obtener el mismo resultado que si la realizases una vez.

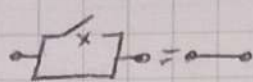
$$\forall a \in B \Rightarrow \begin{cases} a + a + a + \dots = a \\ a \cdot a \cdot a \cdot \dots = a \end{cases}$$

$$\begin{array}{c|c} X + X = X & X \cdot X = X \\ \hline \text{Diagrama de un interruptor en paralelo} & \text{Diagrama de un interruptor en serie} \end{array}$$

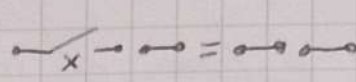
Teorema de los elementos dominantes

$$\forall a \in \mathcal{B} \Rightarrow \begin{cases} a+1=1 \\ a \cdot 0=0 \end{cases}$$

$$x+1=1$$



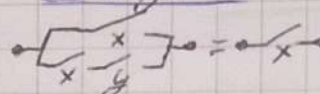
$$x \cdot 0=0$$



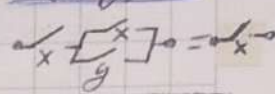
Teorema de absorción

$$\forall a, b \in \mathcal{B} \Rightarrow \begin{cases} a+(a \cdot b)=a \\ a \cdot (a+b)=a \\ a+(\bar{a} \cdot b)=a+b \\ a \cdot (\bar{a}+b)=a \cdot b \end{cases} \quad \text{Absorción parcial}$$

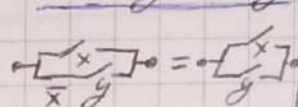
$$x+(x \cdot y)=x$$



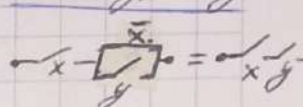
$$x \cdot (x+y)=x$$



$$x+\bar{x} \cdot y=x+y$$

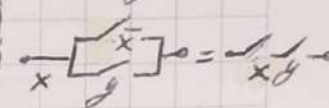
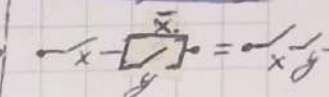
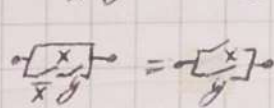
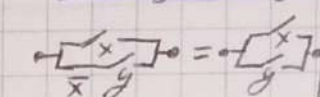


$$x \cdot (\bar{x}+y)=x \cdot y$$



Teorema asociativo

$$\forall a, b, c \in \mathcal{B} \Rightarrow \begin{cases} a+(b+c)=(a+b)+c \\ a \cdot (b \cdot c)=(a \cdot b) \cdot c \end{cases}$$



Teorema involución (doble negación)

$$\forall a \in \mathcal{B} \Rightarrow \bar{\bar{a}}=a$$

Teoremas de Morgan (leyes de Morgan)

$$\forall a, b, c, \dots \in \mathcal{B} \Rightarrow \begin{cases} \overline{a+b+c+\dots} = \bar{a} \cdot \bar{b} \cdot \bar{c} \cdot \dots \\ \overline{a \cdot b \cdot c \cdot \dots} = \bar{a} + \bar{b} + \bar{c} + \dots \end{cases}$$

Función de un álgebra de Boole.

→ Variable lógica o binaria cuyos valores dependen de una expresión algebraica en la cual las variables independientes están ligadas por los tres operadores lógicos (+, ·, -).

→ Estas expresiones algebraicas van a representar las "funciones de transferencia de circuitos lógicos-digitales".

→ Otra forma de representar una función es por medio de su tabla de verdad que nos permite representar el valor que toma la función para cada una de las combinaciones que se pueden formar con las variables independientes.

→ Obtenemos la tabla de verdad de las expresiones algebraicas de salida o de las especificaciones de funcionamiento del circuito.

→ Cada una de las combinaciones de las variables independientes se puede representar por medio del producto lógico de todas ellas, de forma que una variable aparezca en dicho producto en forma directa si toma el valor "1" lógico o en forma negada si toma el valor "0" lógico. Estos productos reciben el nombre de "minitérminos" o "minterms".

Forma canónica. Minterms.

→ La expresión algebraica de una función la podremos obtener, a partir de su tabla de verdad, mediante la suma lógica de los minterms presentes en la función, es decir, por medio de la suma lógica de los minterms para los cuales la función toma valor "1" lógico.

	b	a	F ₁
m ₀ = $\bar{b}\bar{a}$	0	0	0
m ₁ = $\bar{b}a$	0	1	1
m ₂ = $b\bar{a}$	1	0	1
m ₃ = ba	1	1	0

Canónica algebraica

$$F_1(b, a) = \sum_{2^{(k)}} (1, 2) = m_1 + m_2 = \bar{b}a + b\bar{a}$$

Canónica numérica

(*) nº de variables independientes de la función, en este ejemplo 2, "b" y "a".

→ Los minterms son denominados términos canónicos, por tanto, una función representada por sus minterms estará representada en forma canónica (con su máxima redundancia o en su forma más compleja).

Simplificación algebraica

La simplificación de funciones en el Álgebra de Boole se basa en la búsqueda de adyacencias entre términos para poder aplicar a su suma lógica la Propiedad Distributiva seguida de la Ley del Complementario y del elemento neutro.

→ Dos términos o minterms serán adyacentes cuando se diferencien solo en el valor de una de sus variables.

$$cba \neq cb\bar{a}$$

$$cba \neq cb\bar{a}$$

$$\bar{c}a \neq ca$$

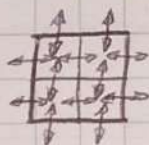
$$ba \neq cb$$

Mapas de Karnaugh (2, 3 y 4 variables)

Como solución a la problemática de la simplificación al existir más de dos términos adyacentes en una expresión algebraica.

Construcción de los mapas de Karnaugh

- 1) Determinar adyacencias entre los minterms de la función
- 2) Tabla o matriz con tantas celdas como minterms tenga la función, consideramos adyacencias verticales y horizontales por los laterales de las celdas.



- 3) Asignamos los minterms a las celdas de la tabla cumpliendo todas las condiciones de adyacencia.
- 4) Determinamos las coordenadas del mapa (posición de las variables).

$$m_0 = 0000$$

$$\dots$$

$$m_{15} = 1111$$

Coordenadas columna

Coordenadas de fila

	ba	00	01	11	10
dc	00	0	1	3	2
	01	4	5	7	6
	11	12	13	15	14
	10	8	9	11	10

"4 variables"

Procedimiento general

1. Rellenamos con "1" lógica aquellas celdas cuyo minterm asociado este presente en la función.
 2. Realizamos el menor número posible de agrupaciones de 8, 4, 2 elementos sin importar que haya solapamientos parciales.
- Los elementos presentes en un recubrimiento equivalen a los términos doblados de la simplificación algebraica.

- Nunca un grupo podrá estar cubierto en su totalidad por otro.
(en dicho caso la expresión tendría redundancia y estaría simplificada al máximo).

Funciones incompletas. Términos inoperantes

- Hay ocasiones en que no importan, o resultan irrelevantes, los valores que tome una función para determinadas combinaciones de las variables independientes, no llegando, siquiera a estar definido el valor que debe tomar la función para estas combinaciones en particular, por no resultar necesario.
- Estas funciones reciben el nombre de "Funciones incompletas", y los términos canónicos o "minterms" que representan dichas combinaciones reciben el nombre de "términos inoperantes" (indeterminados), dado que no condicionan el valor que debe tomar la función para dichas combinaciones.
- La nomenclatura algebraica para representar dichas funciones, en su forma canónica numérica, es la misma que para las funciones completas, pero añadiendo, en este caso, un segundo sumatorio a la función que incluya dichos términos inoperantes, usando como subíndice del sumatorio los símbolos $\emptyset$ o X .

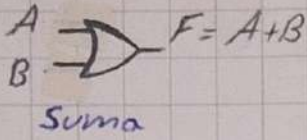
$$F(c, b, a) = \sum_3 (1, 5, 6) + \sum_X (0, 7) \equiv \sum_3 (1, 5, 6) + \sum_{\emptyset} (0, 7)$$

- Los términos inoperantes resultan de suma utilidad cuando estamos simplificando una función, dado que los podemos utilizar como comodines a efectos prácticos, pudiendo considerar que están, o no, presentes en la función con objeto de obtener la expresión algebraica más simplificada.

Puertas lógicas

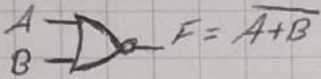
Componente electrónico discreto empleado en el diseño de circuitos lógicos.

OR
+



B \ A	0	1
0	0	1
1	1	1

NOR



Puertas asociativas

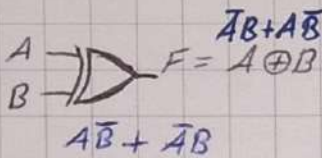
Las que no tienen negado

OR $\Rightarrow \Rightarrow \Rightarrow$

XOR $\Rightarrow \Rightarrow \Rightarrow$

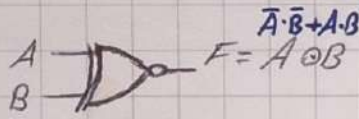
AND $\Rightarrow \Rightarrow \Rightarrow$

XOR
⊕

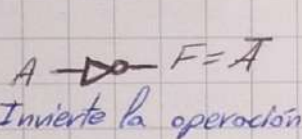


B \ A	0	1
0	0	1
1	1	0

XNOR
⊙

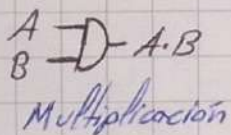


NOT



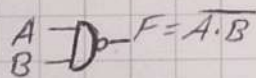
A	F
0	1
1	0

AND
×



B \ A	0	1
0	0	0
1	0	1

NAND



Puertas lógicas. Conjuntos completos.

Un conjunto de puertas lógicas es un "conjunto completo" si nos permite implementar los pares $(+, -)$ o $(\cdot, -)$.

Con un conjunto completo es posible implementar cualquier circuito lógico haciendo uso exclusivo de dicho tipo de puertas.

Una puerta OR y una NOT forman un conjunto completo. Otro conjunto lo forman una puerta AND y una NOT. Con ellas podríamos implementar cualquier circuito lógico.

NOR y NAND por sí solas son un conjunto completo.

Tema 3 Circuitos combinacionales

Circuitos aritméticos

Una proposición puede ser verdadera (1) o falsa (0).

El Álgebra de Boole es un sistema matemático que se utiliza en el análisis y diseño de circuitos electrónicos.

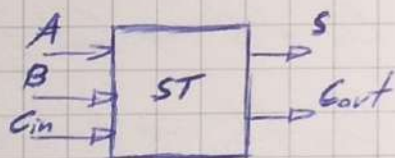
La suma aritmética se reduce a las sumas parciales de los dígitos del mismo orden o peso de ambas cantidades, junto con el dígito de acarreo -"carry"- generado en la suma parcial anterior.

En base 2

La solución para la suma aritmética se reduce al diseño de una etapa sumadora, Sumador Total (ST), que nos permita efectuar las sumas aritméticas parciales de 3 bits del mismo orden o peso.

El primer acarreo de entrada siempre es "0"/masa/tierra.

- Entradas: números a sumar (A, B y C_{in}).
- Salidas: suma y acarreo (S y C_{out}).



A	B	C_{in}	C_{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Inciso simplificación

c	ba	00	01	11	10
0		0	1	1	0
1		1	0	0	1

$$F(c, b, a) = \sum (1, 2, 4, 7) = \bar{c}\bar{b}a + \bar{c}b\bar{a} + c\bar{b}\bar{a} + cba =$$

$$= \bar{c}(\bar{b}a + b\bar{a}) + c(\bar{b}\bar{a} + ba) =$$

$$= \bar{c}(b \oplus a) + c(\overline{b \oplus a}) = c \oplus (b \oplus a) = \underline{c \oplus b \oplus a}$$

Encadenamiento ST

Para la suma de 2, 3, 4... bits:

- Se va sumando el bit con menor peso de ambas números en cada sumador.
- El acarreo de salida del primer sumador hace de acarreo de entrada de el segundo y así sucesivamente.

- El acarreo de salida del último sumador también es una salida

Cuadruple sumador total

Realiza la suma de dos combinaciones de 4 bits con un posible carry inicial.

Para diseñar un sumador de dos combinaciones de "n bits" sería preciso conectar "n etapas" ST en serie (cascada).

Generalización de la multiplicación

$$\begin{array}{r}
 \text{3 bits} \\
 \begin{array}{r}
 a_2 a_1 a_0 / 2 \\
 \times b_2 b_1 b_0 / 2 \\
 \hline
 a_2 b_0 \ a_1 b_0 \ a_0 b_0 \\
 a_2 b_1 \ a_1 b_1 \ a_0 b_1 \ 0 \\
 a_2 b_2 \ a_1 b_2 \ a_0 b_2 \ 0 \ 0 \\
 \hline
 P_5 \ P_4 \ P_3 \ P_2 \ P_1 \ P_0
 \end{array}
 \end{array}$$

$$0 \leq A \text{ y } B \leq 7$$



$$0 \leq A \times B \leq 49$$



6 bits

$$C_{in} = 0 \rightarrow C_{out} = b \cdot a$$

$$C_{in} = 1 \rightarrow C_{out} = b + a$$

$$C_{in} \neq b \rightarrow S = \bar{a}$$

$$C_{in} = b \rightarrow S = a$$

$$S = C_{in} \oplus b \oplus a$$

Sumador total como conjunto completo

C_{in}	b	a	C_{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Extrapolable a cualquiera de las variables de entrada del ST.

$$\text{Si } C_{in} = "0" \rightarrow C_{out} = b \cdot a$$

$$\text{Si } C_{in} = "1" \rightarrow C_{out} = b + a$$

$$\text{Si } C_{in} \neq b \rightarrow S = \bar{a}$$

- Si una de las tres entradas la conectamos a "0" lógico por la salida de carry, C_{out} , obtendremos el producto lógico de las otras dos.

- Si una entrada la conectamos a "1" lógico por la salida de carry, C_{out} , obtendremos la suma lógica de las otras dos variables.

- Si conectamos dos entradas de un ST una a "0" lógico y la otra a "1" lógico, por la salida de suma, S , obtendremos el valor negado de la tercera variable.

Multiplexores

- Permiten enviar la información procedente de diferentes canales de entrada a un único canal de salida.



- Adaptando esta estructura a un circuito lógico:



Si $m=1 \rightarrow n=2$	MXOR 2 canales
Si $m=2 \rightarrow n=4$	MXOR 4 canales
Si $m=4 \rightarrow n=16$	MXOR 16 canales

- Conectará con la salida, z , el canal de entrada cuyo subíndice decimal, i , coincida con la combinación binaria presente en las entradas de selección de canal en cada momento.

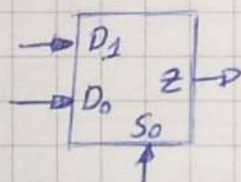
- El circuito dispone de tantos canales de entradas, " n ", como combinaciones se pueden generar con los " m " entradas de selección de canal.

MXOR de 2 canales

S_0	D_1	D_0	z
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

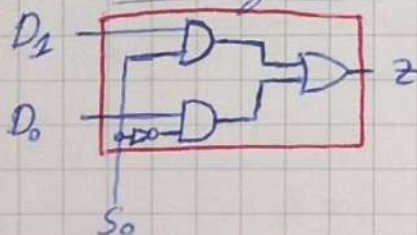
S_0	z
0	D_0
1	D_1

bloque funcional



$$Z = \overline{S_0} D_0 + S_0 D_1$$

Cto. lógico

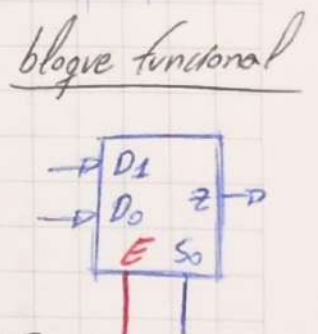


MXOR de 2 canales con entrada "ENABLE" "activa a alto nivel"

E	S_0	D_1	D_0	z
0	X	X	X	0
1	0	0	0	0
1	0	0	1	1
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	0
1	1	1	0	1
1	1	1	1	1

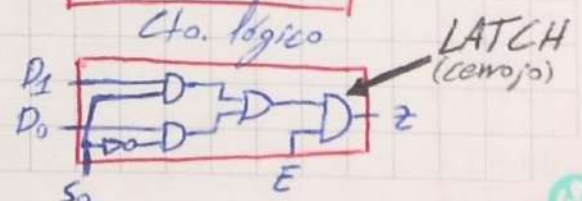
E	S_0	z
0	X	0
1	0	D_0
1	1	D_1

bloque funcional



$$Z = E \overline{S_0} D_0 + E S_0 D_1 = E (\overline{S_0} D_0 + S_0 D_1)$$

Cto. lógico



Codificadores = Circuito combinacional con 2^n entradas y n salidas.

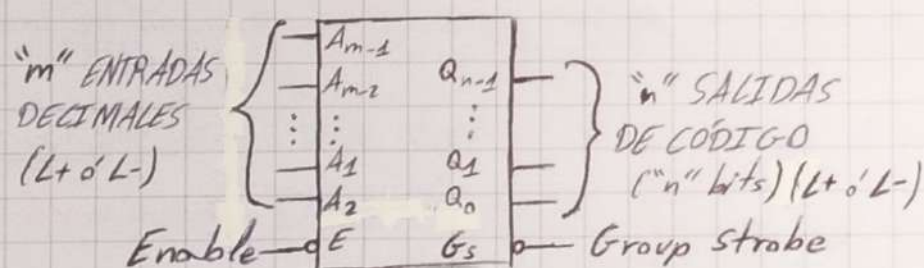
El circuito entrega por las salidas el código codificado con " n " bits el "subíndice decimal" de la entrada activada en cada momento, con relación " $m:n$ " (n° entradas decimales y n° salidas de código) vendrá determinada implícitamente por el código de salida.

Notación: Codificador + Código salida + de " n " bits o de " m " a " n "

Si hay más de una entrada activa se codifican la salida de mayor peso, de mayor prioridad.

Salida Group Strobe (Validación de grupo). Se activa cuando el circuito entrega una combinación de código válida.

Se puede diseñar con entradas y/o salidas activas a nivel alto (L^+), a nivel bajo (L^-) o una combinación de ambas cosas (lógica mixta).



Ejemplo de diseño

Codificador Binario Natural de 2bits (4 a 2)



	A_3	A_2	A_1	A_0	Q_1	Q_0	G_s
$m_0 \equiv 0$	0	0	0	0	0	0	1
$m_1 \equiv 1$	0	0	0	1	0	0	0
$m_2 \equiv 2$	0	0	1	0	0	1	0
$m_3 \equiv 3$	0	0	1	1	0	1	0
$m_4 \equiv 4$	0	1	0	0	1	0	0
$m_5 \equiv 5$	0	1	0	1	1	0	0
$m_6 \equiv 6$	0	1	1	0	1	0	0
$m_7 \equiv 7$	0	1	1	1	1	0	0
$m_8 \equiv 8$	1	0	0	0	1	1	0
$m_9 \equiv 9$	1	0	0	1	1	1	0
$m_{10} \equiv 10$	1	0	1	0	1	1	0
$m_{11} \equiv 11$	1	0	1	1	1	1	0
$m_{12} \equiv 12$	1	1	0	0	1	1	0
$m_{13} \equiv 13$	1	1	0	1	1	1	0
$m_{14} \equiv 14$	1	1	1	0	1	1	0
$m_{15} \equiv 15$	1	1	1	1	1	1	0

	A_3	A_2	A_1	A_0	Q_1	Q_0	G_s
$m_0 \equiv 0$	0	0	0	0	0	0	1
$m_1 \equiv 1$	0	0	0	1	0	0	0
$m_2 \equiv 2$	0	0	1	0	0	1	0
$m_3 \equiv 3$	0	0	1	1	0	1	0
$m_4 \equiv 4$	0	1	0	0	1	0	0
$m_5 \equiv 5$	0	1	0	1	1	0	0
$m_6 \equiv 6$	0	1	1	0	1	0	0
$m_7 \equiv 7$	0	1	1	1	1	0	0
$m_8 \equiv 8$	1	0	0	0	1	1	0
$m_9 \equiv 9$	1	0	0	1	1	1	0
$m_{10} \equiv 10$	1	0	1	0	1	1	0
$m_{11} \equiv 11$	1	0	1	1	1	1	0
$m_{12} \equiv 12$	1	1	0	0	1	1	0
$m_{13} \equiv 13$	1	1	0	1	1	1	0
$m_{14} \equiv 14$	1	1	1	0	1	1	0
$m_{15} \equiv 15$	1	1	1	1	1	1	0

$$Q_1 = \bar{A}_3 \cdot A_2 + A_3$$

$$Q_0 = \bar{A}_3 \cdot \bar{A}_2 \cdot A_1 + A_3$$

$$G_s = \bar{A}_3 \cdot \bar{A}_2 \cdot \bar{A}_1 \cdot \bar{A}_0$$

Decodificadores $\equiv$ Generadores canónicos $\leftarrow$ Entrada de m bits en una salida única de los 2^n disponibles

Función complementaria a los codificadores, el circuito activará la salida cuyo subíndice decimal corresponda con la combinación de " m " bits aplicada sobre los entradas de códigos, relación " $\min$ " (n° de entradas de código y n° de salidas decimales) vendrá determinada implícitamente por el código de entrada.

Notación: Decodificador + Código de entrada + " m " bits ó de " m " a " n ".

Los decodificadores reciben el nombre de "generadores canónicos" porque en cada una de sus salidas decimales se obtiene la expresión algebraica de un "minterm" de " m " variables (correspondiente a las entradas de código) o bien en forma directa (L^+) o negada (L^-).

Esta particularidad sirve para implementar de forma sencilla y rápida funciones lógicas haciendo uso exclusivo de un decodificador del tamaño adecuado y puertas lógicas.

Si el código no es completo ($2^m > n$)

El decodificador se diseña para que desactive todas sus salidas cuando reciba una combinación que no pertenezca al código de entrada.

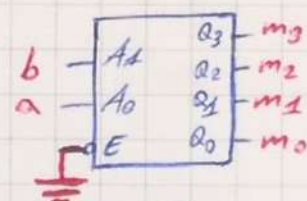
Generación de funciones

$$A_1 = b, A_0 = a \text{ y } E = "0"$$

Generalizando

• Si el decodificador tiene salidas activas a nivel alto basta con utilizar una puerta OR $\Rightarrow$ para sumar las salidas que generen minterms presentes en la función.

• Si el decodificador tiene salidas activas a nivel bajo basta con una puerta NAND $\Rightarrow$ De:



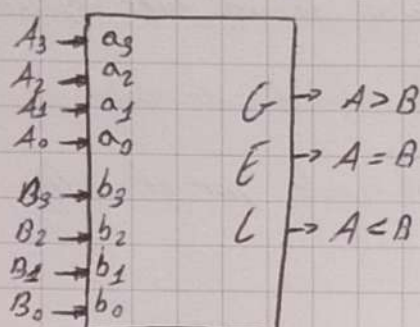
$$\left. \begin{aligned} Q_3 &= \bar{E} A_1 A_0 \\ Q_2 &= \bar{E} A_1 \bar{A}_0 \\ Q_1 &= \bar{E} \bar{A}_1 A_0 \\ Q_0 &= \bar{E} \bar{A}_1 \bar{A}_0 \end{aligned} \right\} \Rightarrow \begin{aligned} Q_3 &= b a = m_3 \\ Q_2 &= b \bar{a} = m_2 \\ Q_1 &= \bar{b} a = m_1 \\ Q_0 &= \bar{b} \bar{a} = m_0 \end{aligned}$$

$$F(b, a) = \sum_2 (0, 1, 3) = m_0 + m_1 + m_3 = Q_0 + Q_1 + Q_3$$

Comparadores de magnitud = Circuito combinatorial que indica la relación entre dos entradas binarias A y B: $A=B$, $A>B$, $A<B$.

Nos permitirá determinar cual de las dos combinaciones de "n bits" es menor o mayor o en su defecto son iguales.

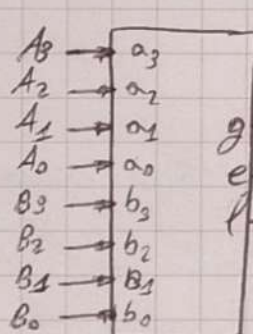
Responden al siguiente bloque funcional



$E \equiv \text{EQUAL}$
 $G \equiv \text{GRAND}$ $L \equiv \text{LESS}$

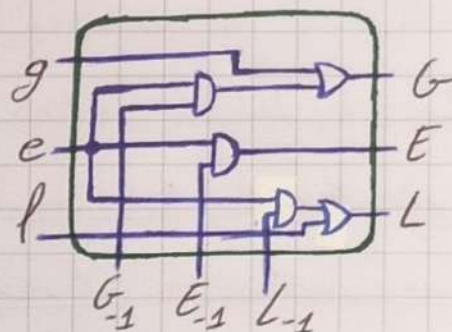
	G	E	L
Si $A > B$	1	0	0
Si $A = B$	0	1	0
Si $A < B$	0	0	1

Los comparadores demuestran mucha lógica en las ampliaciones. Este inconveniente se reduce en parte añadiendo un pequeño bloque al comparador tradicional, expansionor apareciendo el "comparador con entradas de expansión o concatenación".



EXPANSOR ← Solo interviene cuando $A=B$

	g	e	l	G	E	L
Si $A > B$	1	0	0	1	0	0
Si $A = B$	0	1	0	G_{-1}	E_{-1}	L_{-1}
Si $A < B$	0	0	1	0	0	1



$$\begin{aligned} G &= g + eG_{-1} \\ E &= eE_{-1} \\ L &= l + eL_{-1} \end{aligned}$$

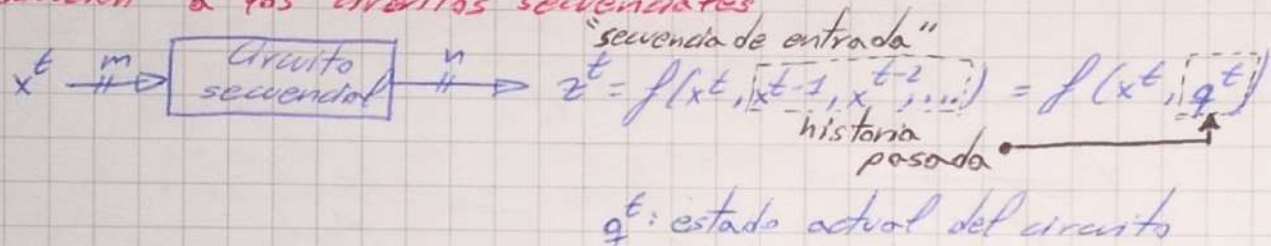
Lógica secuencial

- En un circuito combinatorial la respuesta de salida, z^t , en un instante t , depende única y exclusivamente de la combinación de entrada, x^t , aplicada en ese mismo instante, no influye la historia pasada en su comportamiento.
- La misma combinación de entrada aplicada en distintos instantes de tiempo ($t, t+1, t+2, \dots, t+n$) dará lugar siempre a la misma combinación de salida:

$$z_i^t(x^t) = z_i^{t+1}(x^{t+1}) = \dots = z_i^{t+n}(x^{t+n})$$

- Su comportamiento viene definido por su tabla de verdad

Introducción a los circuitos secuenciales



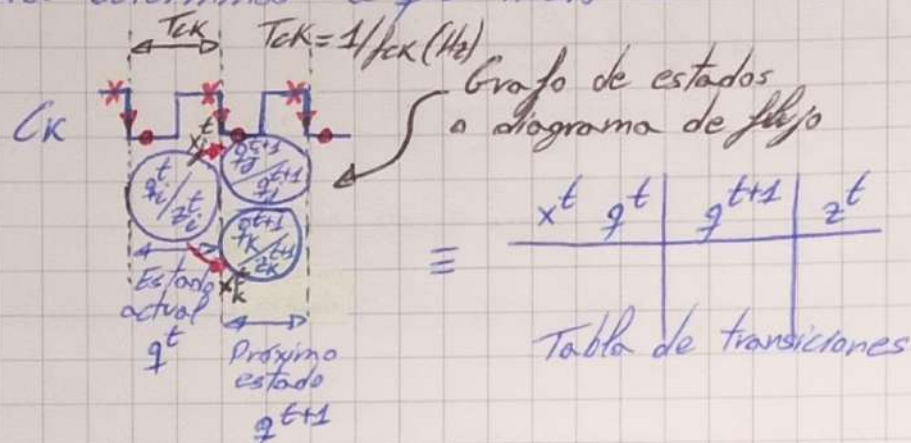
- En un circuito secuencial la respuesta de salida, z^t , en un instante t , no solo vendrá condicionada por la combinación de entrada aplicada en ese mismo instante, x^t , también dependerá de las combinaciones de entrada aplicadas en instantes anteriores, $x^{t-1}, x^{t-2}, \dots$, es decir, influye la historia pasada en su comportamiento, y esa historia pasada, q^t , en cada momento la recordaremos mediante una combinación concreta de " K " vbles. de estado interno

$$q^t \equiv \langle q_{K-1}^t, q_{K-2}^t, \dots, q_1^t, q_0^t \rangle \rightarrow \begin{cases} q_i^t: \text{vble. estado interno} \\ \text{"memoria" de "K" biestables} \end{cases}$$

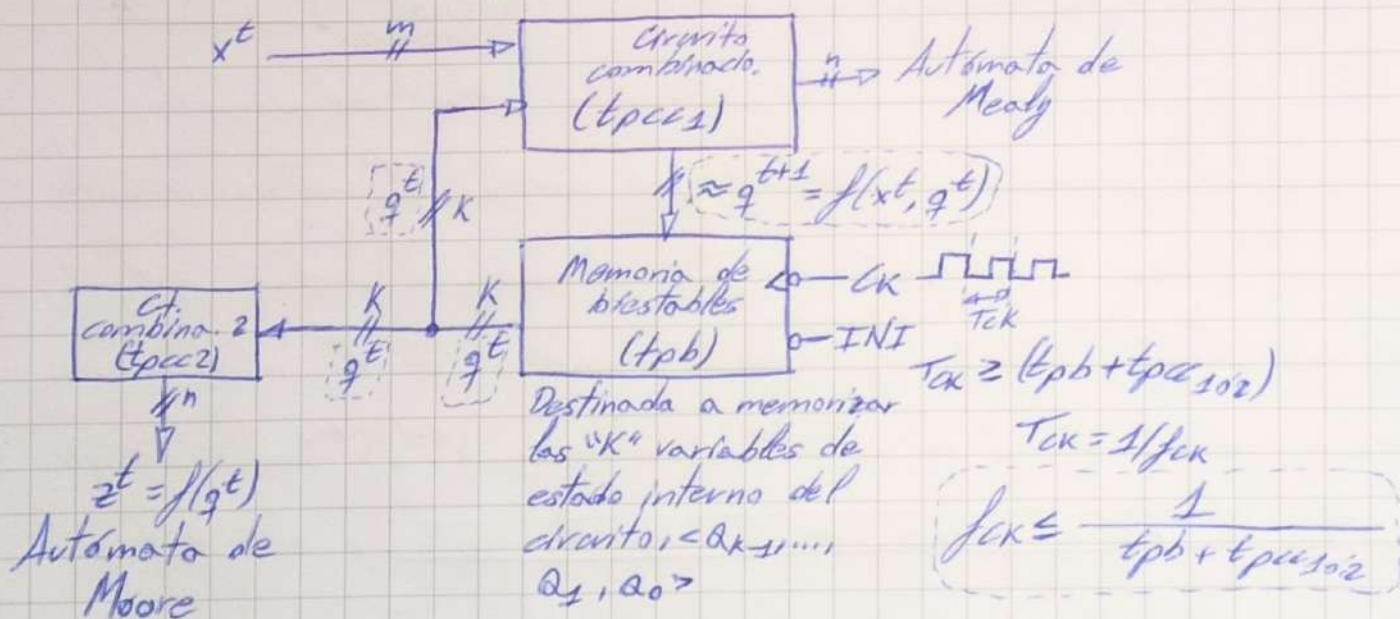
- El comportamiento de un circuito secuencial lo representamos mediante un grafo de estados o diagrama de flip, que nos mostrará la forma en que evoluciona el cto secuencial por los diferentes estados, q^t , del grafo.

Grafo de estados y tabla de transiciones

- Las transiciones o cambios de estado del circuito van a estar controladas por los flancos de bajada o subida de una señal de reloj, CK (clk).
- Dichos flancos marcarán los instantes en los que el circuito registra la información de entrada, x^t , y la combina con su estado actual, q^t , para determinar a qué nuevo estado evoluciona, q^{t+1} .



Circuito secuencial, diagrama de bloques



tpb y $tpcc_{1,2} \equiv$ retardos de propagación

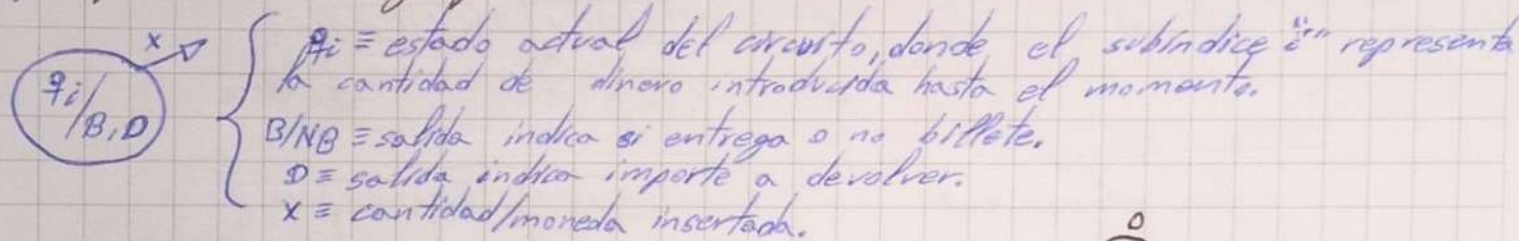
INI $\equiv$ inicialización o preste a cero.

Grafos. Ejemplos

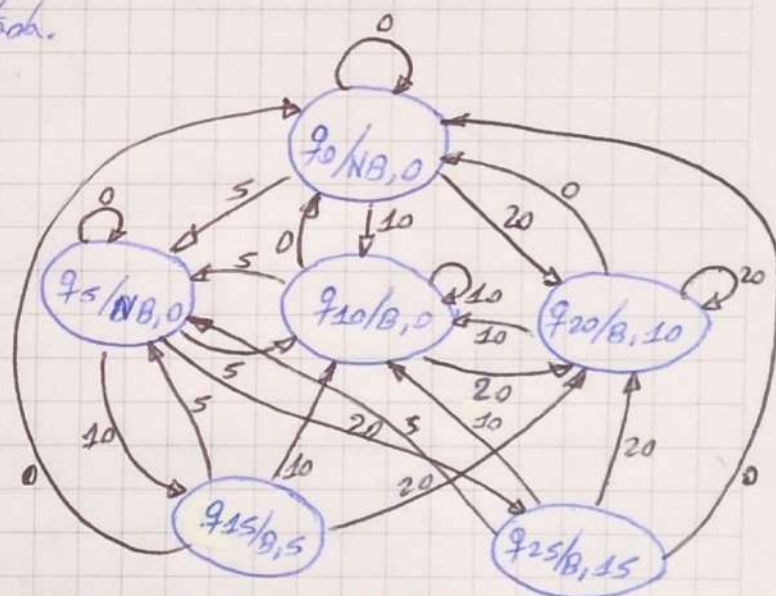
- Obtener el grafo de estados de un automata de Moore que controla el funcionamiento de una máquina del metro. Considere que el precio del billete es de 10 cents y que la máquina admite, exclusivamente monedas de 5, 10, 20 cents.

El automata deberá recordar la cantidad de dinero que lleva introducido. La salida del automata cuando se llegue al importe exacto del billete, o bien se sobrepase este, deberá entregar el billete y efectuar la devolución del cambio que proceda.

Especificación del grafo



El automata tiene que recordar un total de 6 situaciones distintas, (solo una de ellas en cada momento).
 ⇒ 3 bits para el diseño



INI = 0"
 inserción en q_0

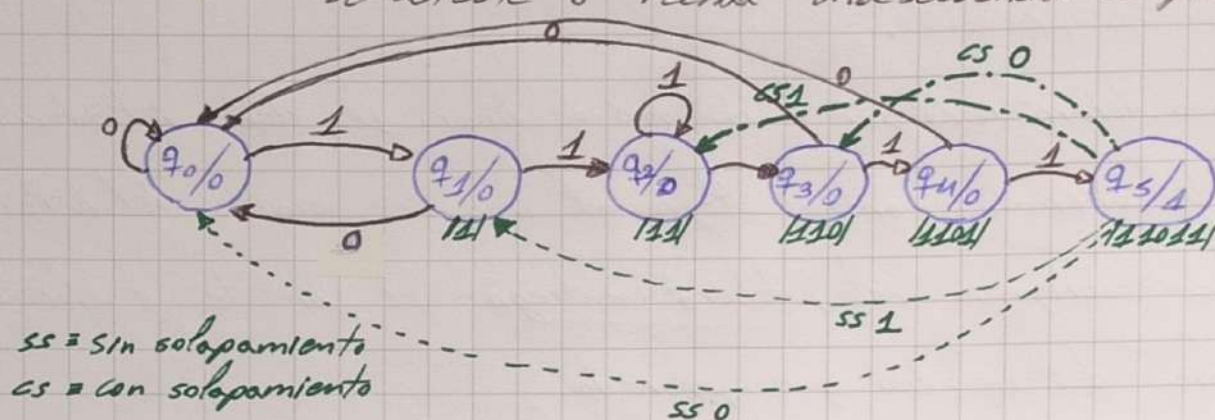
OBTENEMOS su tabla de estados y su tabla de transiciones, que son formas equivalentes al grafo:

$q^t \backslash x^t$	0	5	10	20	z^t
q_0	q_0	q_5	q_{10}	q_{20}	NB, 0
q_5	q_5	q_{10}	q_{15}	q_{25}	NB, 0
q_{10}	q_0	q_5	q_{10}	q_{20}	B, 0
q_{15}	q_0	q_5	q_{10}	q_{20}	B, 5
q_{20}	q_0	q_5	q_{10}	q_{20}	B, 10
q_{25}	q_0	q_5	q_{10}	q_{20}	B, 15

x^t	q^t	q^{t+1}	z^t
0	q_0	q_0	NB, 0
0	q_5	q_5	NB, 0
0	q_{10}	q_0	B, 0
0	q_{15}	q_0	B, 5
20	q_{20}	q_{20}	B, 10
20	q_{25}	q_{20}	B, 15

Obtener el grafo de estados de un autómata de Moore que detecte el patrón .../11011/... en una secuencia infinita de bits que recibe en serie y sincronizada con la señal de reloj Clk .

q_i/z El subíndice indicará el número de bits de la secuencia que se han recibido hasta el momento. $z=1$ cuando se detecte o reciba una secuencia completa.



Sin solapamiento

x^t ...	0	1	1	0	1	1	0	0	0	...
z^t ...	0	0	0	0	0	1	0	0	0	0

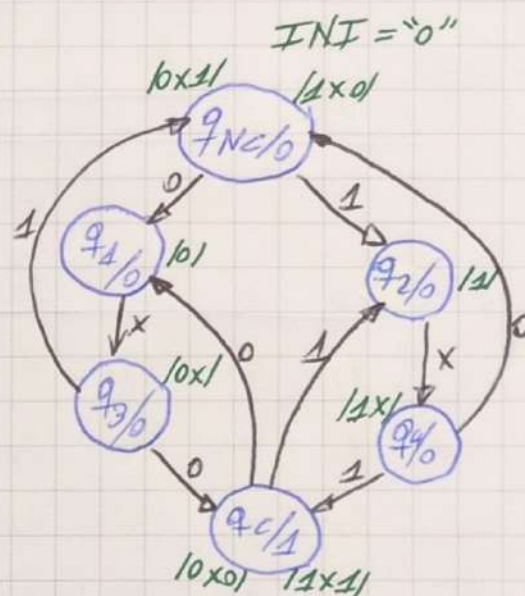
Con solapamiento

x^t ...	0	1	1	0	1	1	0	1	1	0	0	...
z^t ...	0	0	0	0	0	1	0	0	1	0	0	...

Obtener el grafo de estados de un autómata de Moore que recibe por una línea serie y sincronizados con la señal de reloj paquetes de 3 bits y que con el 3er bit de cada paquete el autómata deberá indicar si dicho paquete corresponde o no a un copiaro.

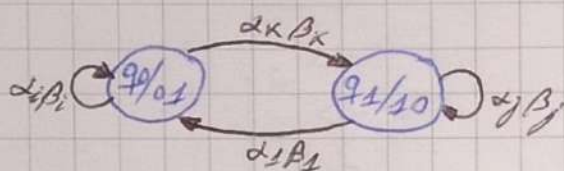
x^t ...	000	110	101	111	011	...
z^t ...	00	1	000	00	1	000

El autómata tiene que recordar un total de 6 situaciones distintas
 $\Rightarrow$ 3 vbles de estado interno
 $\Rightarrow$ 3 biestables para su diseño.



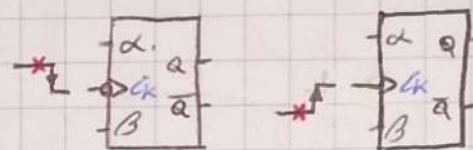
Biestables M/s (Master/Slave)

- Biestable ideal: autómata de Moore que dispone de dos salidas Q y $\bar{Q}$ que en todo momento toman valores complementarios. Su grafo de estados está formado por dos datos, q_0 y q_1 , estables (se mantendrá de forma permanente en cualquiera de ellos hasta que se le fuerce a bascular al otro estado por medio de sus entradas α y β).

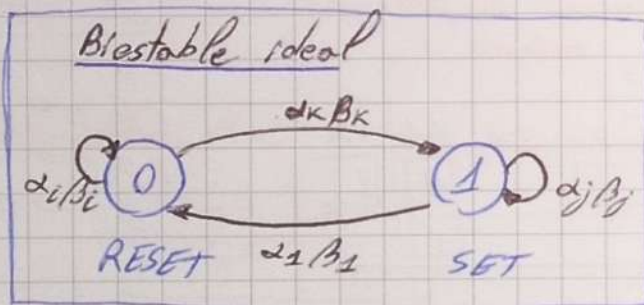


$$q_0 = \begin{cases} Q = 0 \\ \bar{Q} = 1 \end{cases}$$

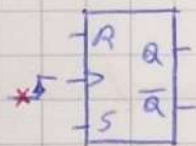
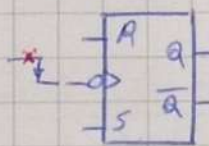
$$q_1 = \begin{cases} Q = 1 \\ \bar{Q} = 0 \end{cases}$$



α y β solo son operativas en el instante antes del flanco (M/S)



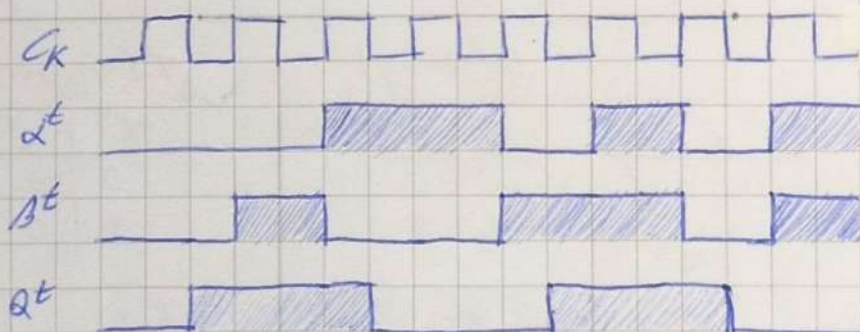
Biestable R-S NOR M/s



R^t	S^t	Q^{t+1}
0	0	Q^t
0	1	1
1	0	0
1	1	0(x)

Ejemplo de funcionamiento

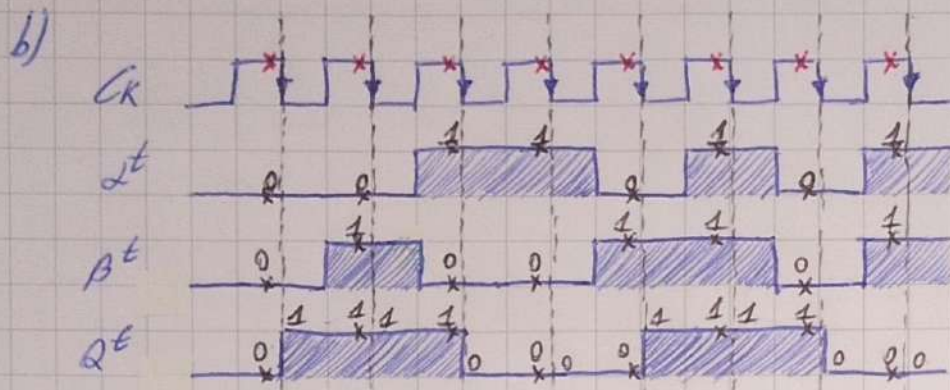
El cronograma pertenece a un biestable α - β



a) Es un M/S?

b) Obtén su grafo y tabla de transiciones

a) En el cronograma se observan cambios en las salidas del biestable con los flancos de bajada de la señal de reloj, se trata de un biestable síncrono por flanco o Master/slave.



Q^t	β^t	Q^{t+1}
0	0	$\bar{Q}^t$
0	1	1
1	0	0
1	1	Q^t

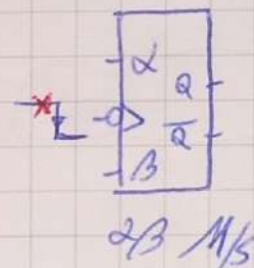
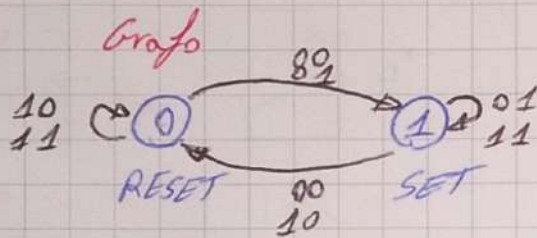


Tabla de Transiciones

Q^t	β^t	Q^t	Q^{t+1}
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

Ecuación de próximo estado

β^t	Q^t	00	01	11	10
0	1			1	1
1				1	

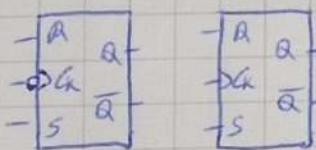
$$Q_{AB}^{t+1} = \bar{Q}^t \cdot \bar{\beta}^t + \beta^t \cdot Q^t$$

Tabla de excitación

$Q^t \rightarrow Q^{t+1}$	Q^t	β^t
0 $\rightarrow$ 0	1	x
0 $\rightarrow$ 1	0	x
1 $\rightarrow$ 0	x	0
1 $\rightarrow$ 1	x	1

Biestables comerciales M/S (Master/slave)

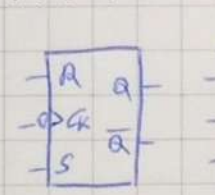
RS NOR



R-S NAND

R^t	S^t	Q^{t+1}
0	0	Q^t
0	1	1
1	0	0
1	1	Q^t

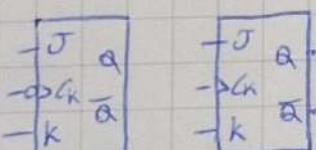
R-S NAND



R-S NAND

R^t	S^t	Q^{t+1}
0	0	1(x)
0	1	0
1	0	1
1	1	Q^t

J-K ($J \equiv S$) ($K \equiv R$)

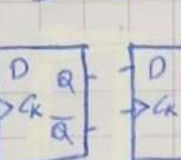


J-K

J^t	K^t	Q^{t+1}
0	0	Q^t
0	1	0
1	0	1
1	1	$\bar{Q}^t$

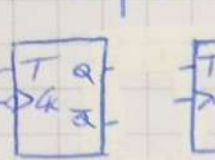
D (Delay)

D^t	Q^{t+1}
0	0
1	1



T (Trigger)

T^t	Q^{t+1}
0	1
1	0



Biestables comerciales R-S NOR (parámetros)

R^t	S^t	Q^{t+1}
0	0	Q^t
0	1	1
1	0	0
1	1	0(x)

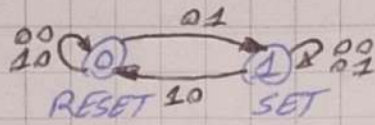


Tabla de transiciones

R^t	S^t	Q^t	Q^{t+1}
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	0(x)
1	1	1	0(x)

$R^t \backslash S^t$	00	01	11	10
0		1	1	1
1			x	x

$$Q^{t+1} = S^t + \bar{R}^t Q^t$$

$$R^t \cdot S^t = "0"$$

Ecuación de próximo estado

Tabla de excitación

$Q^t \rightarrow Q^{t+1}$	R^t	S^t
0 $\rightarrow$ 0	x	0
0 $\rightarrow$ 1	0	1
1 $\rightarrow$ 0	1	0
1 $\rightarrow$ 1	0	x

Biestables R-S NAND (parámetros)

R^t	S^t	Q^{t+1}
0	0	1(x)
0	1	0
1	0	1
1	1	Q^t

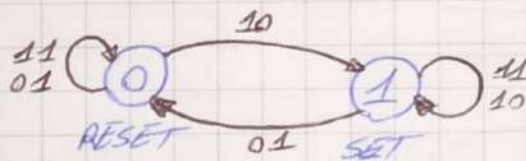


Tabla de transiciones

R^t	S^t	Q^t	Q^{t+1}
0	0	0	1(x)
0	0	1	1(x)
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	1

$R^t \backslash S^t$	00	01	11	10
0	x	x		
1	1	1	1	

$$Q^{t+1} = \bar{S}^t + R^t Q^t$$

$$R^t + S^t = "1"$$

Ecuación de próximo estado

Tabla de excitación

$Q^t \rightarrow Q^{t+1}$	R^t	S^t
0 $\rightarrow$ 0	x	1
0 $\rightarrow$ 1	1	0
1 $\rightarrow$ 0	0	1
1 $\rightarrow$ 1	1	x

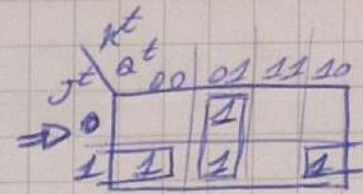
Bistable J-K (parámetros)

J-K ($J=S$ y $K=R$)

J^t	K^t	Q^{t+1}
0	0	Q^t
0	1	0
1	0	1
1	1	$\bar{Q}^t$

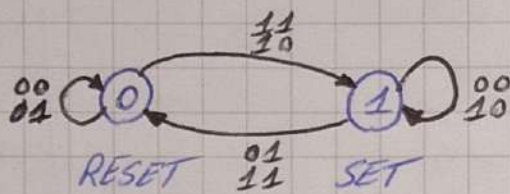
Tabla de transiciones

J^t	K^t	Q^t	Q^{t+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0



$$Q^{t+1}_{JK} = JQ^t + \bar{K}Q^t$$

Ecuación de próximo estado.



Grafo de estados

$\Rightarrow$

$Q^t \rightarrow Q^{t+1}$	J^t	K^t
0 $\rightarrow$ 0	0	x
0 $\rightarrow$ 1	1	x
1 $\rightarrow$ 0	x	1
1 $\rightarrow$ 1	x	0

Tabla de excitación

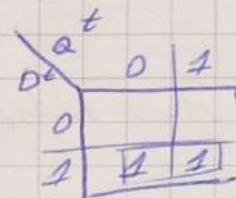
Bistable D (parámetros)

D^t	Q^{t+1}
0	0
1	1

Tabla de transiciones

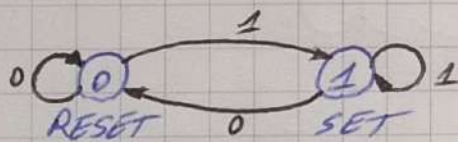
D^t	Q^t	Q^{t+1}
0	0	0
0	1	0
1	0	1
1	1	1

$\Rightarrow$



$$Q^{t+1}_D = D^t$$

Ecuación de próximo estado.



Grafo de estados

$\Rightarrow$

$Q^t \rightarrow Q^{t+1}$	D^t
0 $\rightarrow$ 0	0
0 $\rightarrow$ 1	1
1 $\rightarrow$ 0	0
1 $\rightarrow$ 1	1

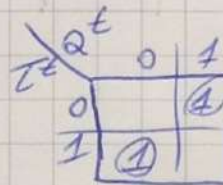
Tabla de excitación

Bistable T (parámetros)

T^t	Q^{t+1}
0	Q^t
1	$\bar{Q}^t$

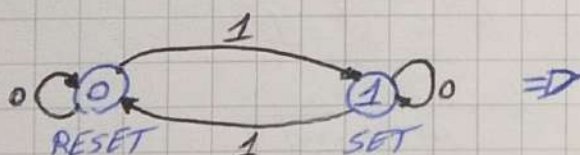
Tabla de transiciones

T^t	Q^t	Q^{t+1}
0	0	0
0	1	1
1	0	1
1	1	0



$$Q^{t+1}_T = T \oplus Q^t$$

Ecuación de próximo estado.



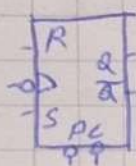
$\Rightarrow$

$Q^t \rightarrow Q^{t+1}$	T^t
0 $\rightarrow$ 0	0
0 $\rightarrow$ 1	1
1 $\rightarrow$ 0	1
1 $\rightarrow$ 1	0

Tabla de excitación

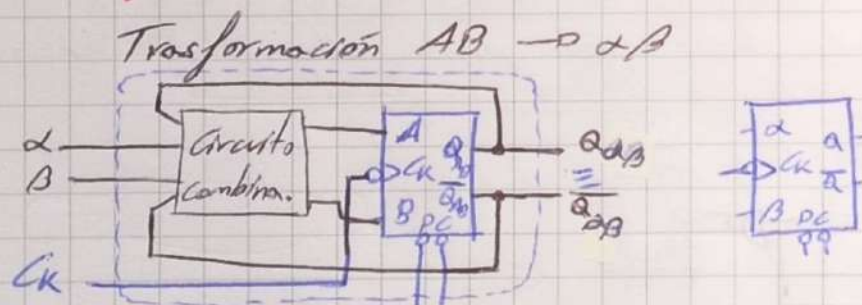
Entradas asincrónicas Preset y Clear

Son dos "entradas de naturaleza asincrónica" (independientes de la señal de reloj), generalmente activas a nivel bajo, "con prioridad sobre las entradas síncronas", cuyo cometido es inicializar el biestable a "0" - Clear - ó a 1 - Preset - de forma inmediata. P y C solo afectan al esclavo (slave).



C	P	CK	R^t	S^t	Q^{t+1}
0	0	x	x	x	1
0	1	x	x	x	0
1	0	x	x	x	1
1	1	$\bar{C}$	0	0	Q^t
1	1	$\bar{C}$	0	1	1
1	1	$\bar{C}$	1	0	0
1	1	$\bar{C}$	1	1	0(x)

Transformación de biestables



Método de la tabla de excitación del biestable A-B

① Tabla de transición biestable $\alpha\beta$

α^t	β^t	Q^t	Q^{t+1}	A^t	B^t
0	0	0	0	x	0
0	0	1	0	1	x
...	...	...	...	...	...
1	1	1	1	0	x

② A^t y $B^t = f(\alpha^t, \beta^t, Q^t)$

✓ este método siempre admite solución

✓ obligatorio si A-B es un R-S.

Método de igualación de las ecuaciones de próximo estado.

$$Q_{AB}^{t+1} = Q_{AD}^{t+1} \rightarrow (1) Q_{AB}^t + (2) \bar{Q}_{AB}^t = (3) Q_{AB}^t + (4) \bar{Q}_{AB}^t$$

Obtenemos un sistema de ecuaciones cuya resolución nos permitirá obtener las expresiones algebraicas de A^t y B^t .

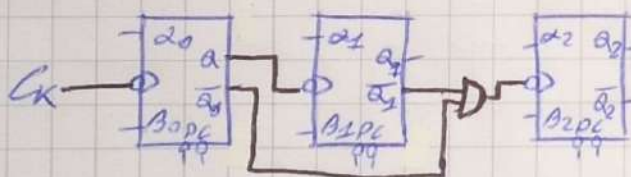
$$\left. \begin{array}{l} (1) = (3) \\ (2) = (4) \end{array} \right\} \Rightarrow A^t \text{ y } B^t = f(Q_{AB}^t, \bar{Q}_{AB}^t)$$

- ✓ el método es aplicable si el sistema de ecuaciones admite solución.
- ✓ no válido si A-B es un R-S (ecuaciones de próximo estado incompletas).
- ✓ poco recomendable si A-B es un T (sistema de ecuaciones complejo).
- ✓ ideal si el biestable A-B es un J-K o un D.

Análisis y síntesis de circuitos secuenciales

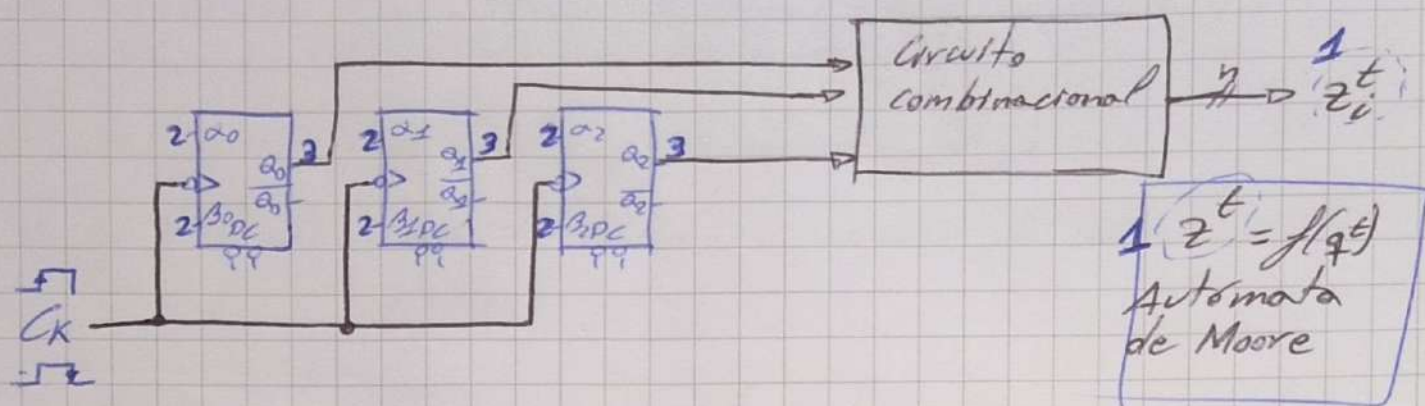
En la práctica nos podemos encontrar con dos tipos de circuitos secuenciales:

- ① Circuitos asincrónicos: algún biestable generalmente el primero, toma su sincronismo de la señal de reloj externa, C_k , el resto se encuentran sincronizados desde algún punto interno del circuito.



Dado que los biestables están sincronizados con distintas bases de tiempo la única posibilidad de análisis es por medio de un "cronograma", analizando para cada biestable, a nivel individual y en el momento en que recibe cada flanco activo de reloj, la situación particular de sus entradas para ver si bascula o se mantiene en el mismo estado. No existe metodología general de diseño para este tipo de circuitos (la excepción de los contadores asincrónicos).

② Circuitos síncronos: todos los biestables están sincronizados con la señal de reloj externa, ck .



En esta situación si es factible optar la siguiente metodología de análisis (en sentido inverso al diseño o síntesis del circuito):

- 1 Ecuaciones de salida del circuito → Automata de Moore.
- 2 Ecuaciones de entrada o excitación, α_i y β_i de todos los biestables
- 3 Ecuaciones de próximo estado, $Q_i^{t+1} = f(Q_i, \beta_i)$ de todos los biestables
- 4 Tabla de transiciones
- 5 Codificación de los estados
- 6 Grafo de estados → función del automata?

Los pasos 2 y 3 se pueden realizar mediante tablas si las ecuaciones son complejas de manejar.

Contadores

Conceptos previos

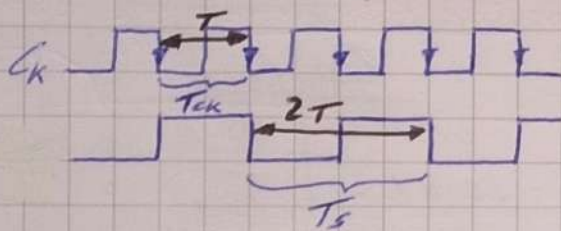
Es un circuito secuencial que pasa por determinados estados con un orden establecido, es un autómata de Moore $z_i^t = Q_i^t$ (las salidas del sistema son las salidas de los biestables).

Son autómatas de grafo cíclico y sin ramificaciones:

donde M es el módulo del contador o número de estados por los que evoluciona el contador.

Si $M=2^n$ es un contador de módulo máximo, donde n es el número de bits (o biestables) del contador.

Contador $\equiv$ Divisor de frecuencia $\Rightarrow$ Módulo (M) $\equiv$ Factor de división

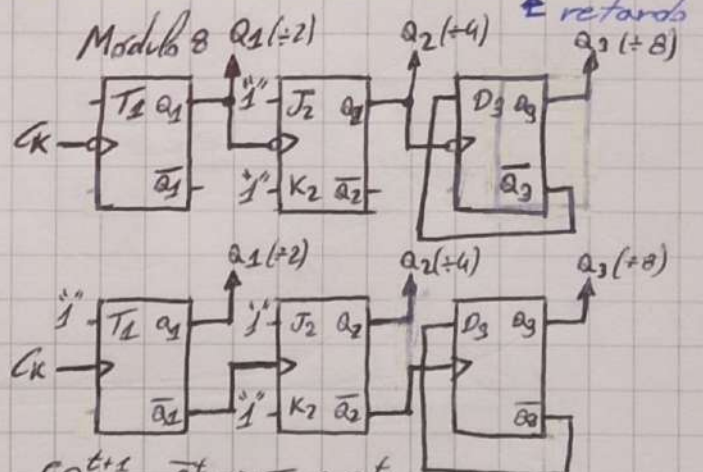
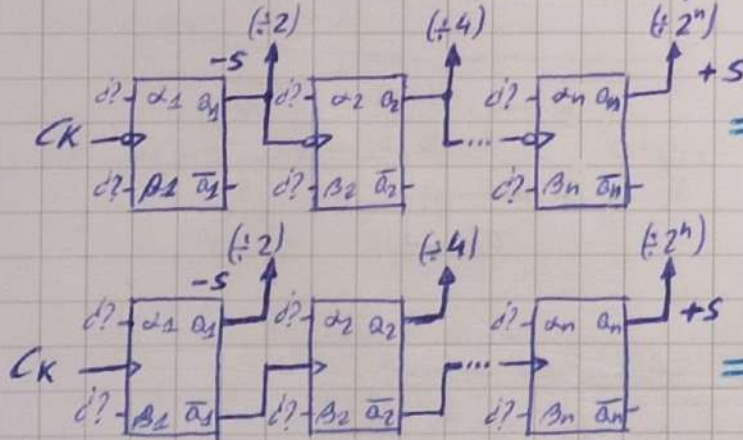


$$T_s = 2 \cdot T_{ck}$$

Contadores asincrónicos UP de módulo máximo ($M=2^n$)

La señal de reloj se aplica a uno de los biestables,

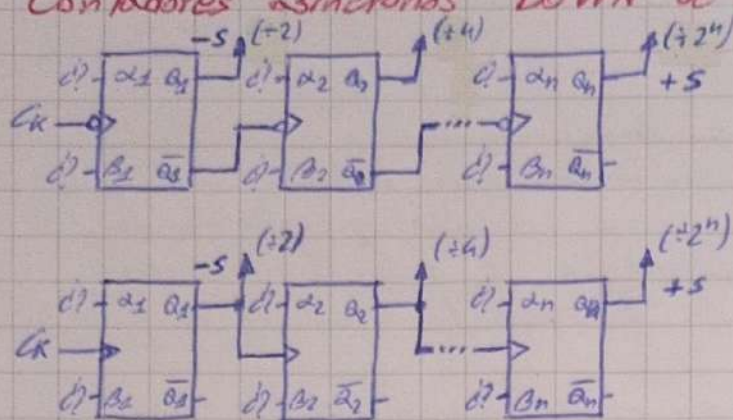
siendo la señal de reloj del resto de los biestables distintas.



$$d_i, \alpha_i, \beta_i / Q_i^{t+1} = \overline{Q_i^t}$$

$$\begin{cases} Q_1^{t+1} = \overline{Q_1^t} \Rightarrow T_1 \oplus Q_1^t = \overline{Q_1^t} \Rightarrow T_1 = "1" \\ Q_2^{t+1} = \overline{Q_2^t} \Rightarrow J_2 \overline{Q_2^t} + K_2 Q_2^t \Rightarrow J_2 = K_2 = "1" \\ Q_3^{t+1} = \overline{Q_3^t} \Rightarrow D_3^t = \overline{Q_3^t} \end{cases}$$

Contadores asíncronos DOWN de módulo máximo ($M=2^n$)

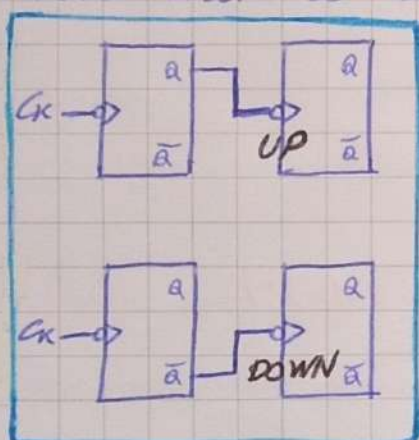


$$Q_i^{t+1} = \bar{Q}_i^t$$

Contadores asíncronos UP-DOWN

Se combinan ambos contadores y se selecciona el sentido de cuenta por medio de la entrada $\bar{U}/D$.

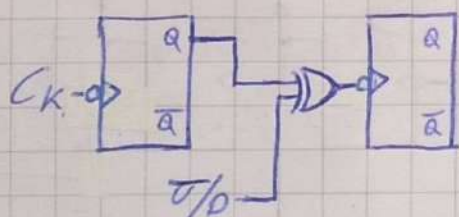
- Para biestables síncronos por flanco de bajada:



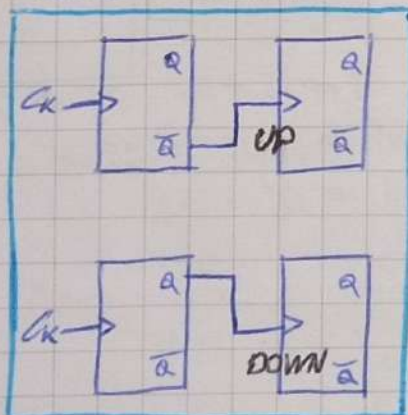
$\bar{U}/D$	Ck_{i+1}
$UP \equiv 0$	Q_{i-1}
$DOWN \equiv 1$	$\bar{Q}_{i-1}$

$$Ck_{i+1} = \bar{U}/D \cdot Q_{i-1} + U/D \cdot \bar{Q}_{i-1} = U/D \oplus Q_{i-1}$$

$$Z_{MXOR} = \bar{S}_0 \cdot D_0 + S_0 \cdot D_1$$



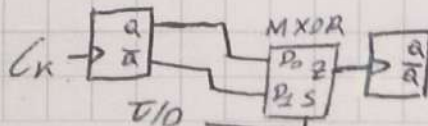
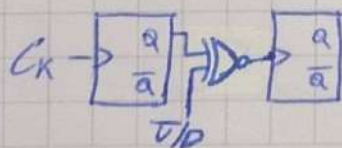
- Para biestables síncronos por flanco de subida.



$\bar{U}/D$	Ck_{i+1}
$UP \equiv 0$	$\bar{Q}_{i-1}$
$DOWN \equiv 1$	Q_{i-1}

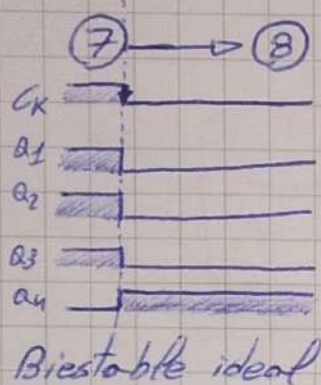
$$Ck_{i+1} = \bar{U}/D \cdot \bar{Q}_{i-1} + U/D \cdot Q_{i-1} = U/D \oplus Q_{i-1}$$

$$Z_{MXOR} = \bar{S}_0 \cdot D_0 + S_0 \cdot D_1$$



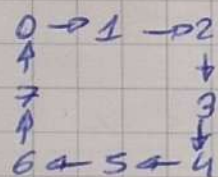
Contadores asincrónicos, "falsos estados (marasitos)"

Un biestable no cambia hasta que lo hayan hecho los que le preceden en la cadena, los biestables tienen un "tiempo de establecimiento" no nulo, aparecen entonces transiciones que afectan a dos o mas biestables, una serie de falsos estados de muy corta duración, esto hace que se descarten estos contadores en aplicaciones prácticas donde esto suponga un inconveniente.



Contador UP síncrono de módulo máximo ($M=2^n$)

Módulo 8



Q_3^t	Q_2^t	Q_1^t	Q_3^{t+1}	Q_2^{t+1}	Q_1^{t+1}
0	0	0	0	0	1
0	0	1	0	1	0
0	1	0	0	1	1
0	1	1	1	0	0
1	0	0	1	0	1
1	0	1	1	1	0
1	1	0	1	1	1
1	1	1	0	0	0

$$Q_3^{t+1} = \overline{Q_3^t} (\overline{Q_2^t} \overline{Q_1^t}) + Q_3^t (\overline{Q_2^t} \overline{Q_1^t})$$

$$Q_2^{t+1} = Q_2^t \oplus Q_1^t$$

$$Q_1^{t+1} = \overline{Q_1^t}$$

Se quiere con J-K

$$Q_{J_1 K_1}^{t+1} = Q_1^{t+1}$$

$$J_1 \overline{Q_1^t} + \overline{K_1} Q_1^t = \overline{Q_1^t}$$

$$J_1 = K_1 = "1"$$

$$Q_{J_2 K_2}^{t+1} = Q_2^{t+1}$$

$$J_2 \overline{Q_2^t} + \overline{K_2} Q_2^t = Q_2^t \oplus Q_1^t = Q_2^t \cdot \overline{Q_1^t} + \overline{Q_2^t} \cdot Q_1^t$$

$$K_2 = Q_1^t$$

$$J_2 = Q_1^t$$

$$J_2 = K_2 = Q_1^t$$

$$Q_{J_3 K_3}^{t+1} = Q_3^{t+1}$$

$$J_3 \overline{Q_3^t} + \overline{K_3} Q_3^t = Q_3^t (\overline{Q_2^t} \overline{Q_1^t}) + \overline{Q_3^t} (\overline{Q_2^t} \overline{Q_1^t})$$

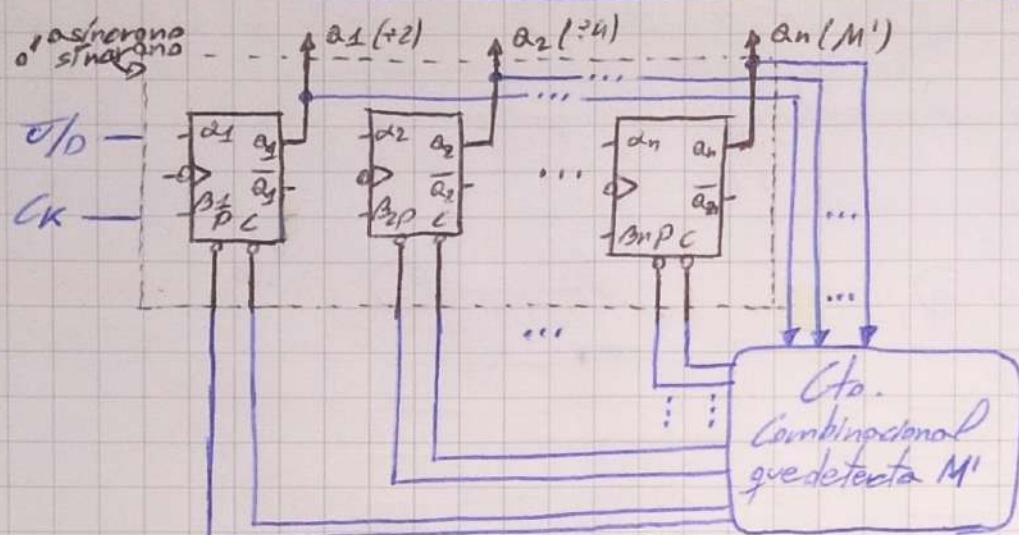
$$J_3 = \overline{Q_2^t} \overline{Q_1^t}$$

$$K_3 = \overline{Q_2^t} \overline{Q_1^t}$$

$$J_3 = K_3 = \overline{Q_2^t} \overline{Q_1^t}$$

Contadores de módulo $M' < M = 2^n$

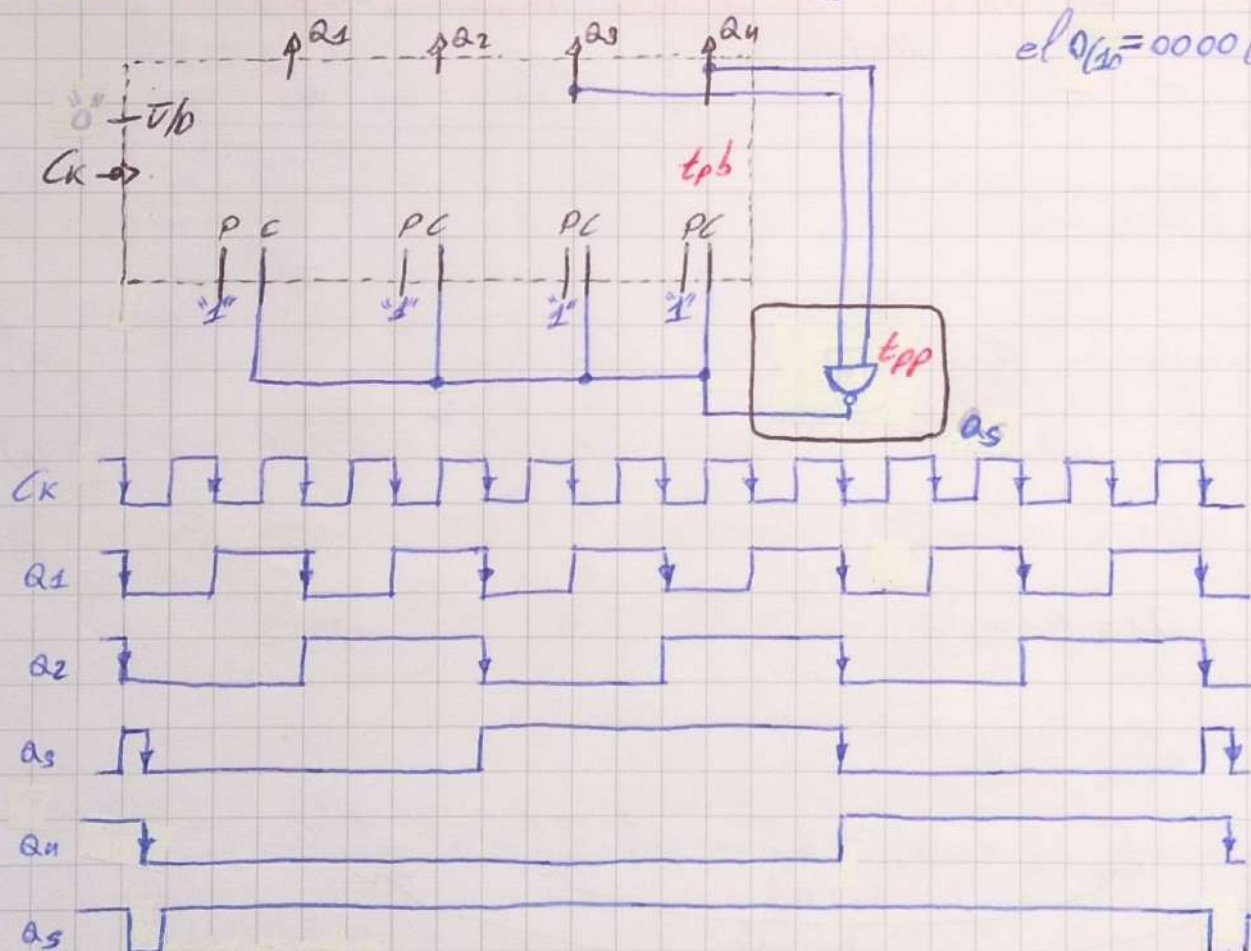
Partiendo de un contador asíncrono o síncrono de módulo máximo, $M = 2^n$, el diseño se reduce a detectar, por medio de un circuito combinatorial, el nuevo estado M' activando, en ese preciso momento, las entradas asíncronas Preset y Clear de cada biestable, inicializando el contador en el estado deseado.



Contador UP de módulo 12

$$M' = 12 < 2^4$$

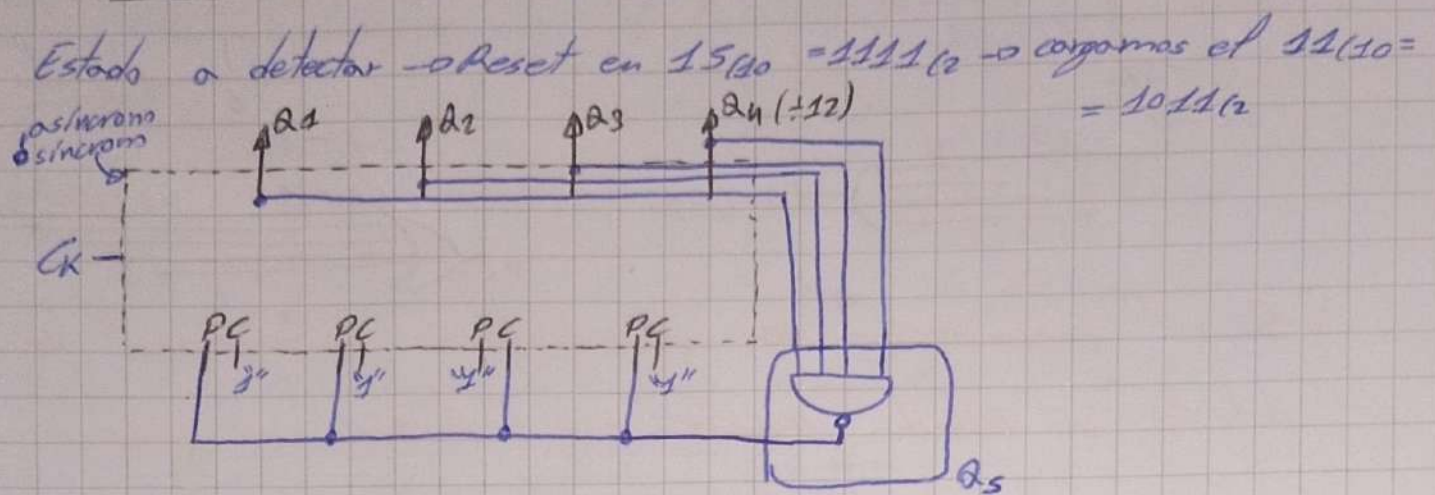
Estado a detectar $M' = 12_{10} = 1100_2 = 11XX_2$ y cargamos el $0_{10} = 0000_2$



$$t_{pb} + t_{pp}$$

(ans $\ll t_{CK} \ll t_s$)

Contador DOWN de módulo 12 (0-11)

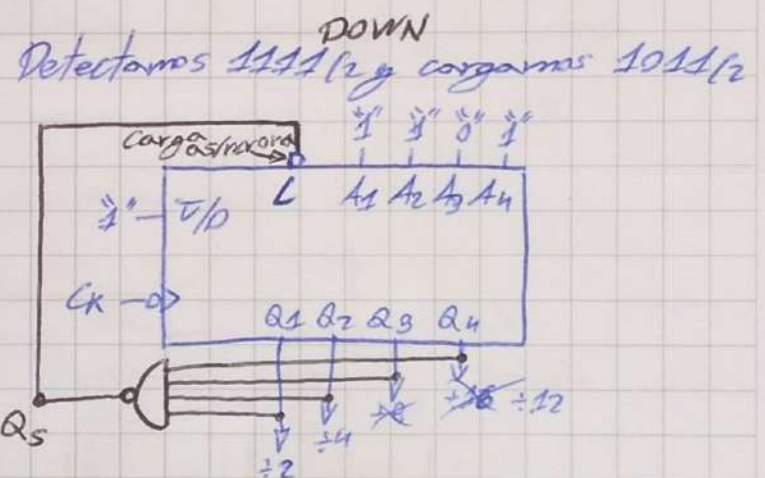
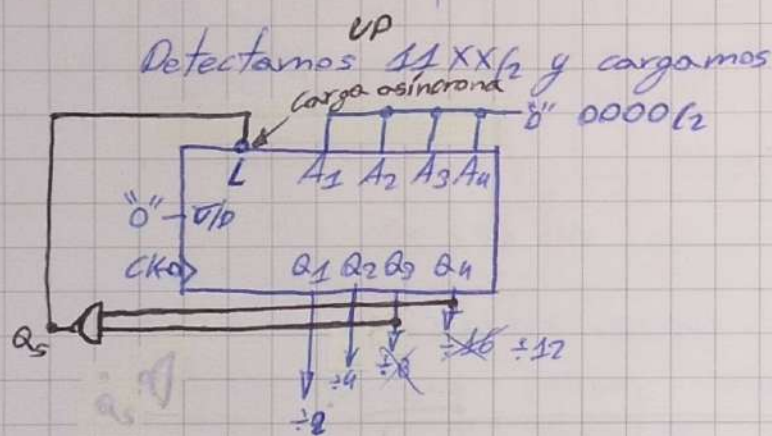
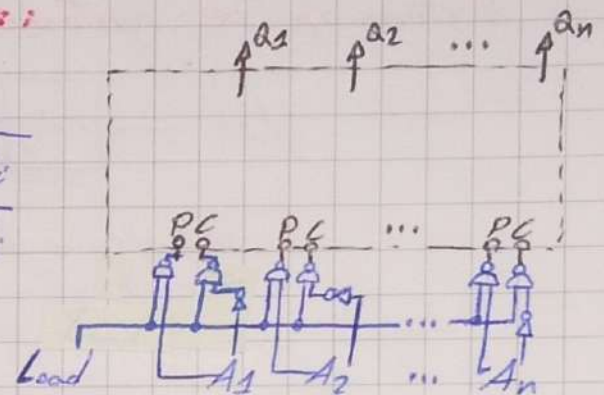


Alternativa

Entradas de carga -LOAD- asíncronas:

Load	A _i	P _i	C _i
0	0	1	0
0	1	1	0
1	X	1	1

$$\Rightarrow \begin{cases} P_i = \overline{L_a A_i} \\ C_i = \overline{L_a} \overline{A_i} \end{cases}$$

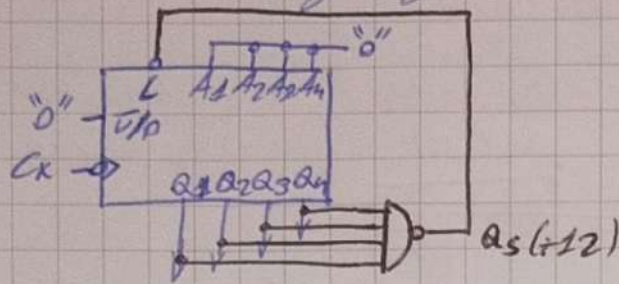


Entradas de carga -LOAD- síncronas

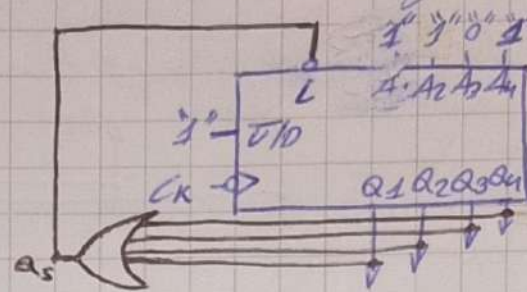
Load	U/D	Q ₄ ^t	Q ₃ ^t	Q ₂ ^t	Q ₁ ^t	Q ₄ ^{t+1}	Q ₃ ^{t+1}	Q ₂ ^{t+1}	Q ₁ ^{t+1}
0	X	X	X	X	X	A ₄	A ₃	A ₂	A ₁
1	0	CONTADOR UP				UP			
1	1	CONTADOR DOWN				DOWN			

Solo los contadores síncronos disponen de entradas de carga síncronas.

UP ^{7 No entiendo}
 Detectamos 1011(2) y cargamos 0000(2)

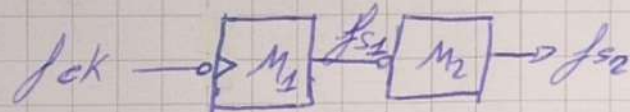


DOWN ^{7 No entiendo}
 Detectamos 0000(2) y cargamos 1011(2)



!!! Si el mecanismo de inicialización es "síncrono" se detecta un estado menos con relación a un mecanismo de inicialización "asíncrono" !!!

Extensión de contadores/divisores



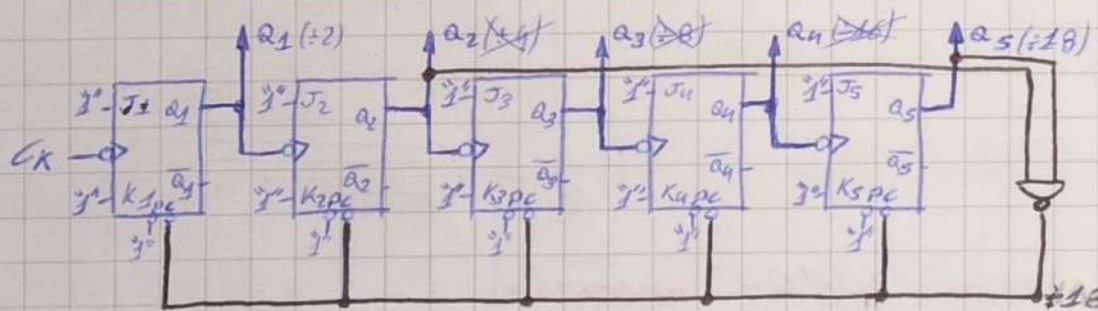
$$fs1 = fck / M1 \Rightarrow fs2 = fs1 / M2 = fck / (M1 \times M2)$$

Apoyandose en esta propiedad hemos diseñado los contadores "asíncronos" vistos con anterioridad.

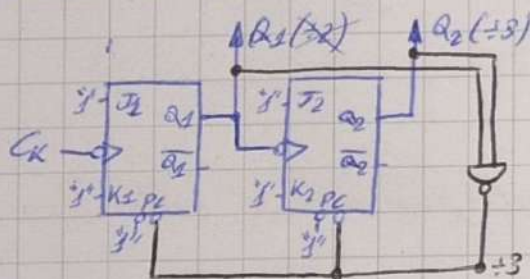
Divisor de frecuencia con factores 3 y 18.

Reducir un contador de módulo 32 a 18. $M = 18 < 2^5$

Estado a detectar $\rightarrow M = 18(10) = 10010(2) \equiv 1 \times 1 \times 1 \times 1 \times 2$



Al no ser 3 potencia entera de dos no puede estar presente en el contador de módulo 18.

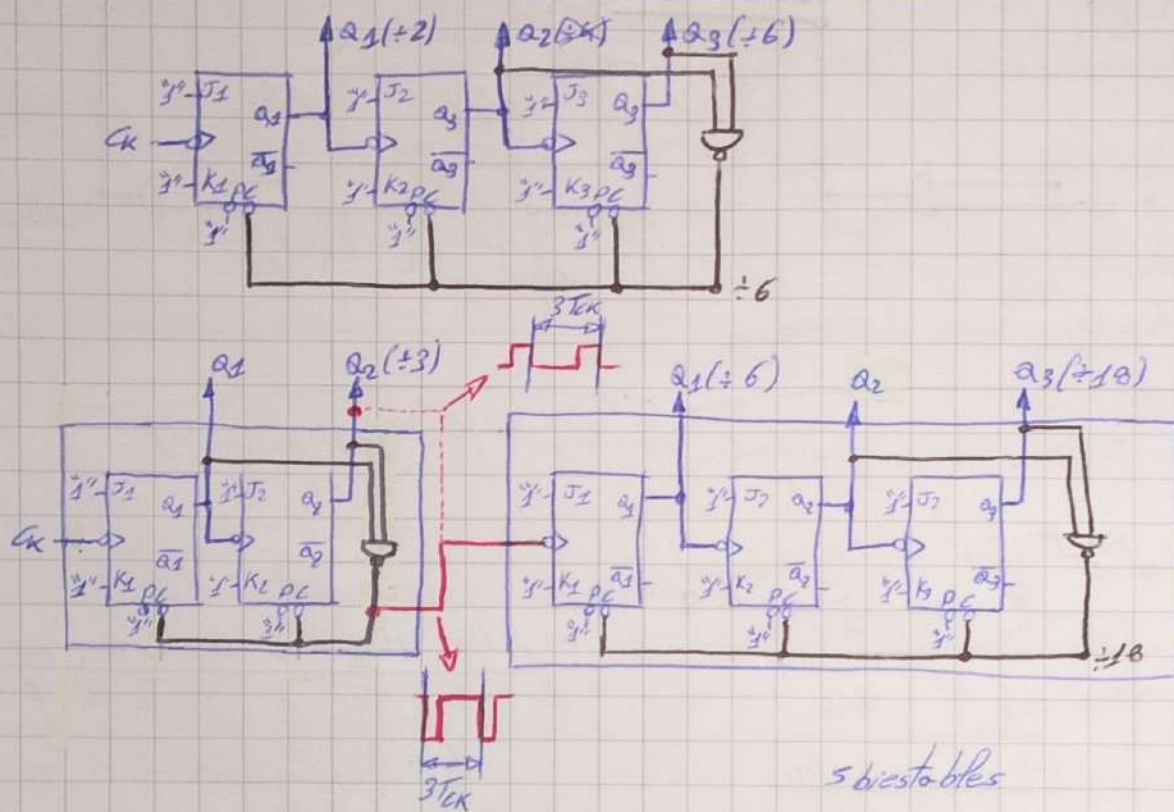


Previsando (5+2) biestables para implementar el diseño mediante las estructuras generales.

Por este procedimiento no generan binario natural los contadores
 ↳ Descomposición factorial (producto de factores) si tenemos en cuenta

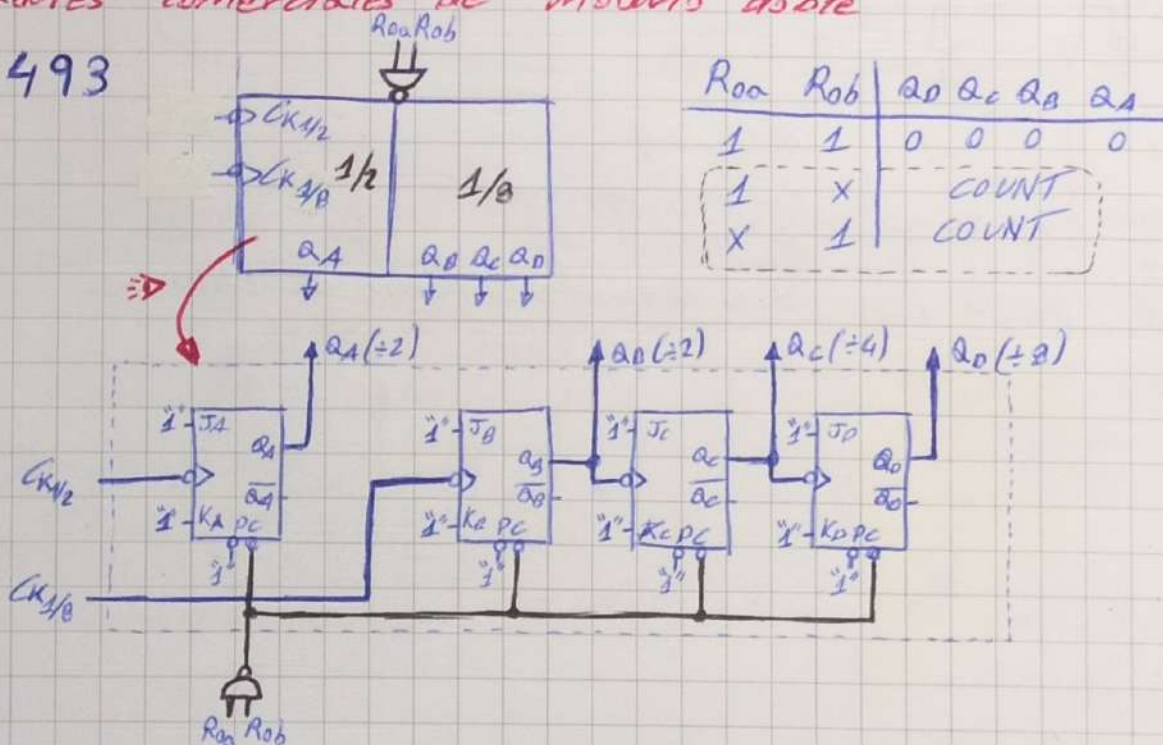
que al concatenar contadores se verifica el producto de factores o módulos y que $18 = 3 \cdot 6$, bastaría con concatenar dos contadores de módulos 3 y 6 respectivamente para obtener los factores buscados:

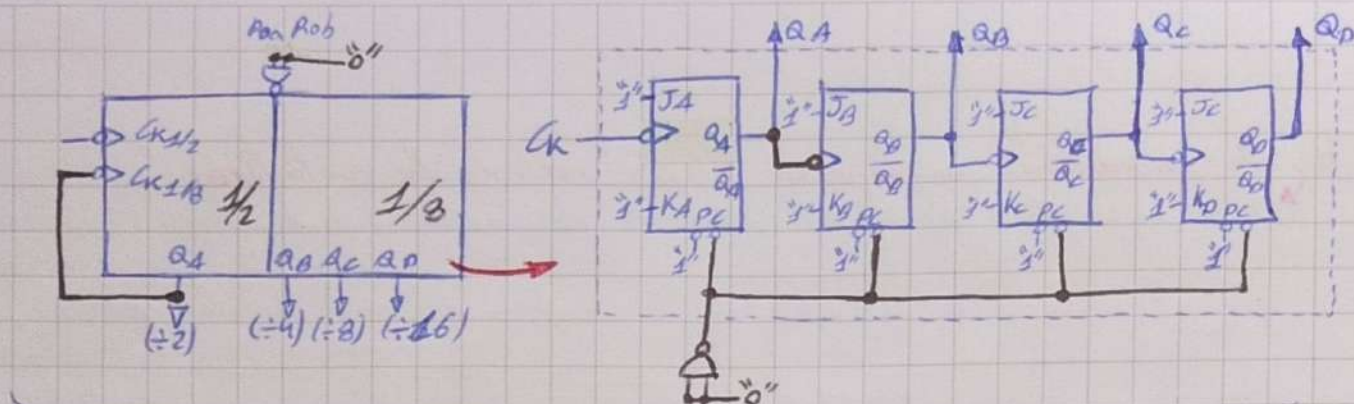
$M = 6 \times 2^3 \rightarrow$ Contador CP de 3 bits o módulo 8.

$$M = 6(10) = 110(2) \equiv 11 \times (2) \text{ estado a detector}$$


Contadores comerciales de método doble

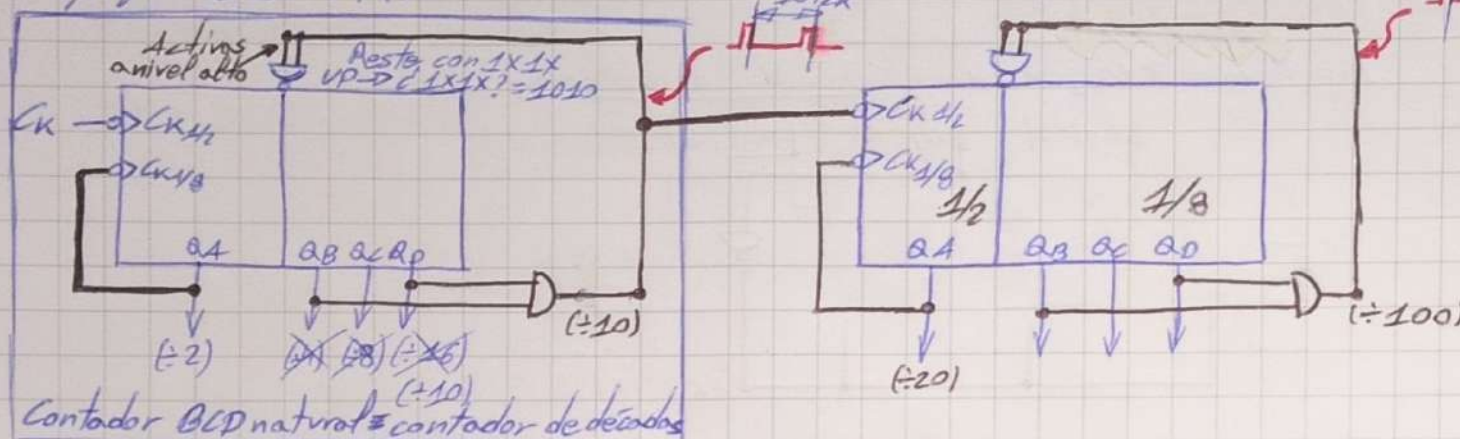
7493





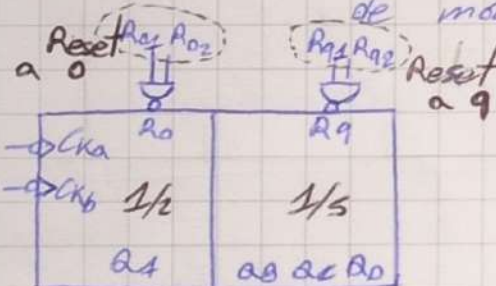
Contador Hexadecimal
o módulo 16 (cuenta en binario Natural)

Ejemplo de uso 7493:



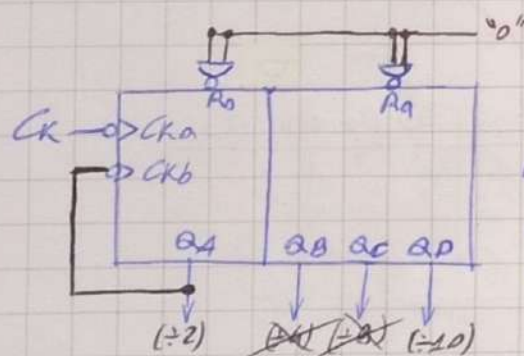
Contador Decimal (en BCD Natural)
de módulo 100.

7490



RA1	RA2	RB1	RB2	QA	QB	QC	QD
1	1	X	X	1	0	0	1
0	X	1	1	0	0	0	0
X	0	1	1	0	0	0	0

Resto de combinaciones COUNT



UP (M=10)
Contador decimal
de Módulo 10
BCD Natural $\equiv$ Contador de Décadas.

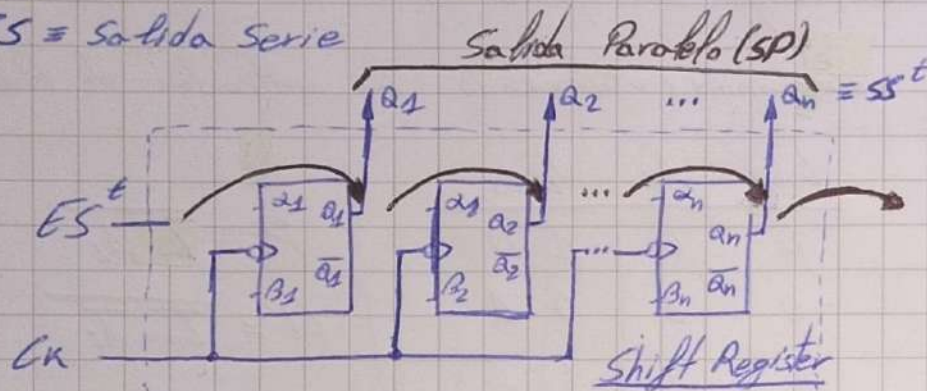
Registros

Registro: "array" o cadena de biestables conectados de forma síncrona, destinados a almacenar información, permitiéndonos al mismo tiempo realizar desplazamiento en serie sobre la misma - Shift Register -.

Es el tipo más básico de memoria, equivalente a una organización de $1 \times n$, una palabra de n bits.

ES $\equiv$ Entrada Serie

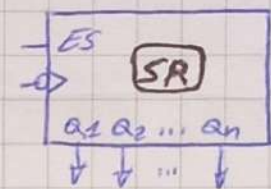
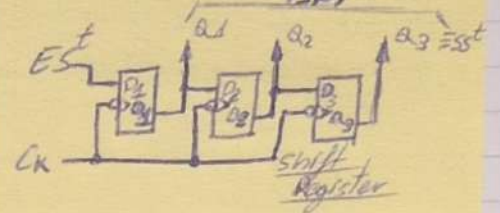
SS $\equiv$ Salida Serie



$$d\alpha_i \text{ y } \beta_i \mid \begin{cases} \alpha_1^{t+1} = ES^t \\ \alpha_i^{t+1} = \alpha_{i-1}^t \quad (i > 1) \end{cases}$$

Equivalente a transformar $\alpha \beta \rightarrow D$

En la práctica y comercial se utiliza el biestable D.



Ejemplo: Registro de Desplazamiento de 3 bits con biestables J-K.

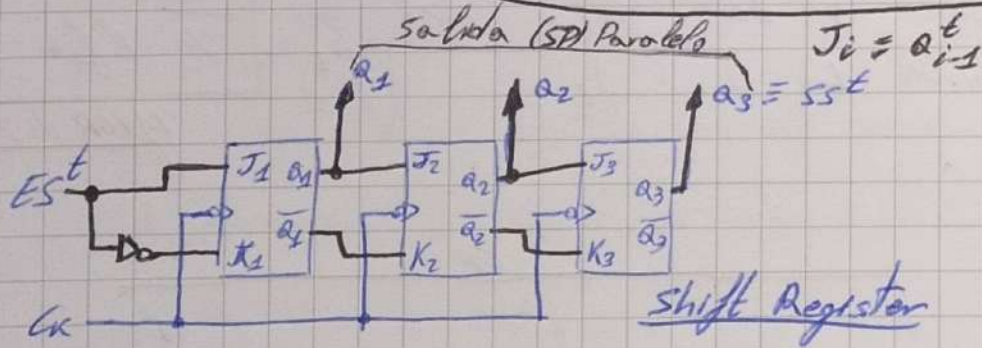
$$dJ_i \text{ y } K_i \mid \begin{cases} \alpha_1^{t+1} = ES^t \\ \alpha_i^{t+1} = \alpha_{i-1}^t \quad (i > 1) \end{cases}$$

$$\alpha_1^{t+1} = J_1 \bar{\alpha}_1^t + \bar{K}_1 \alpha_1^t = ES^t = ES^t (\alpha_1^t + \bar{\alpha}_1^t) = ES^t \alpha_1^t + ES^t \bar{\alpha}_1^t$$

$$K_1 = \bar{ES}^t, J_1 = ES^t$$

$$\alpha_i^{t+1} = J_i \bar{\alpha}_i^t + \bar{K}_i \alpha_i^t = \alpha_{i-1}^t = \alpha_{i-1}^t (\alpha_i^t + \bar{\alpha}_i^t) = \alpha_{i-1}^t \alpha_i^t + \alpha_{i-1}^t \bar{\alpha}_i^t$$

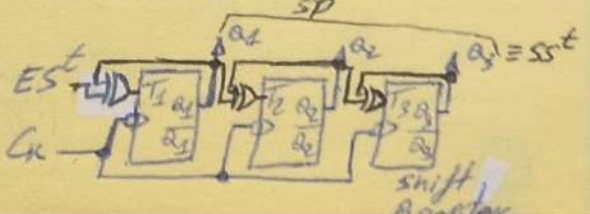
$$K_i = \bar{\alpha}_{i-1}^t, J_i = \alpha_{i-1}^t$$



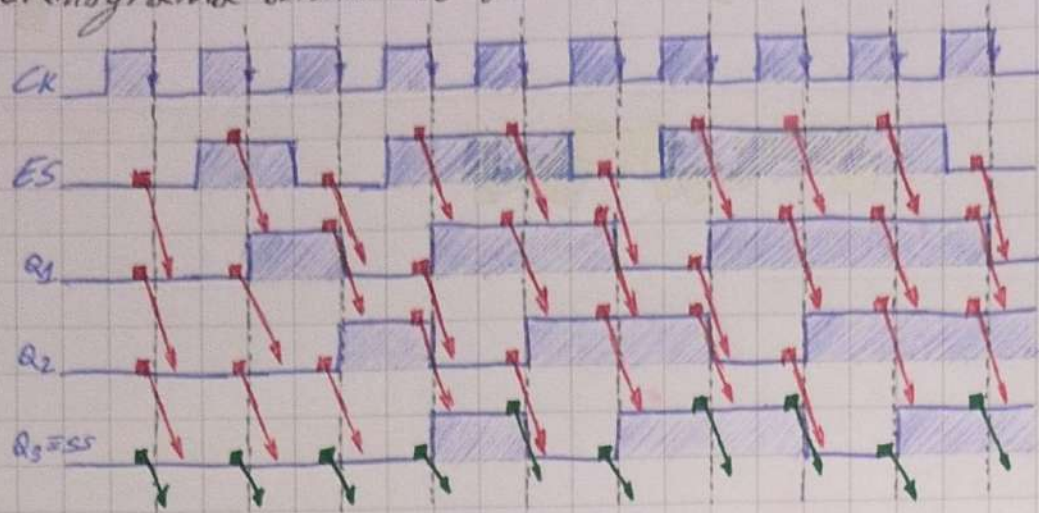
El ejemplo anterior con biestables tipo T quedaría:

$$d\alpha_i \mid \alpha_1^{t+1} = ES^t \text{ y } \alpha_i^{t+1} = \alpha_{i-1}^t \quad (i > 1)$$

$$\alpha_1^{t+1} = T_1 \oplus \alpha_1^t = ES^t \oplus (T_1 \oplus \alpha_1^t) \oplus \alpha_1^t = ES^t \oplus \alpha_1^t \oplus T_1 \oplus \alpha_1^t = ES^t \oplus \alpha_1^t$$



Cronograma obtenido de los diseños anteriores:

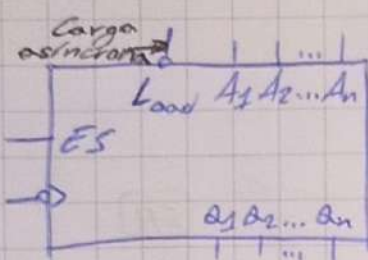


bits transferidos al siguiente biestable en cada flanco de reloj.

bits que salen del registro por la salida Q_3 y se pierden.

Registros con entradas de carga en paralelo

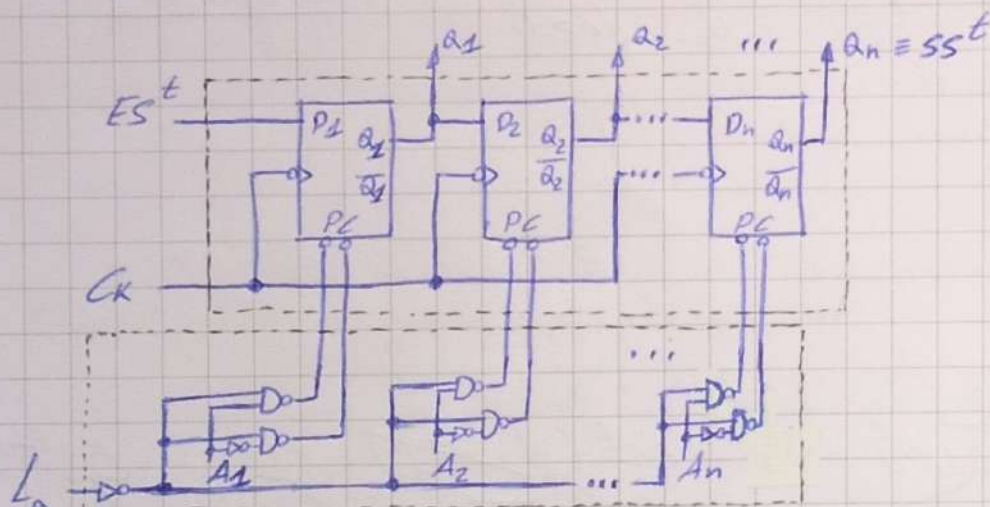
Entradas de carga asincrónicas: utilizando las entradas de Preset y Clear.



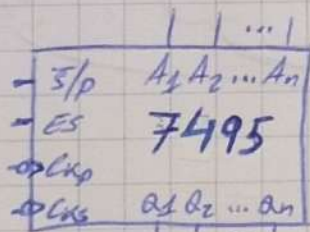
Load	A_i	P_i	C_i
0	0	1	0
0	1	0	1
1	X	1	1

$$P_i = \overline{L_a} A_i$$

$$C_i = \overline{L_a} \overline{A_i}$$



Entradas de carga asincrónicas



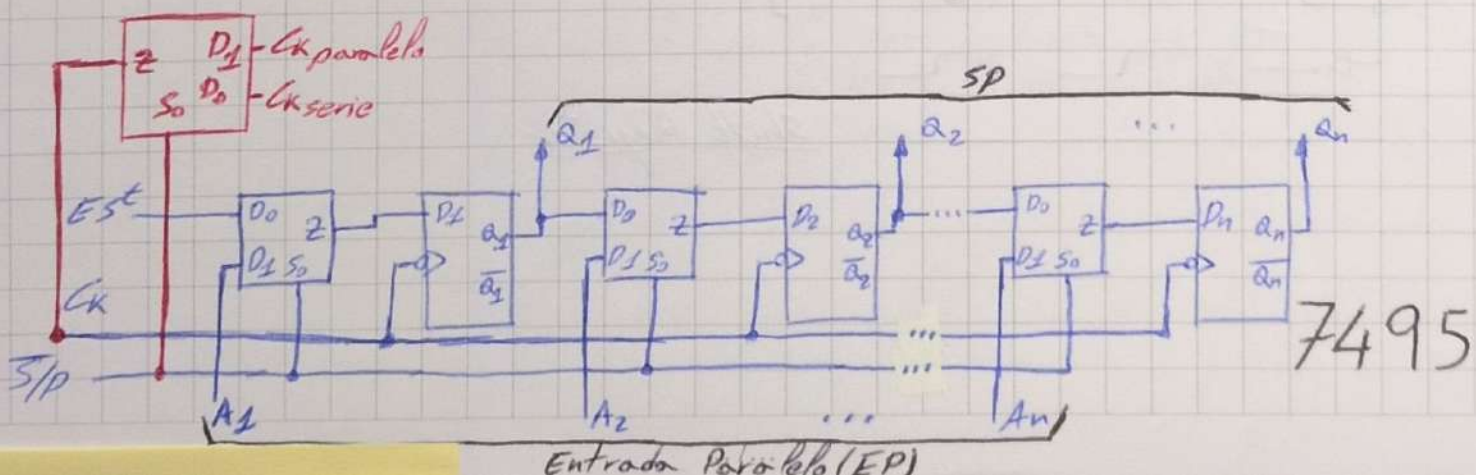
	S/p	Q_1^{t+1}	Q_{i+1}^{t+1}
Serie 0	ES^t	Q_i^t	Q_{i-1}^t
Paralelo 1	A_1	A_i	

$$Q_1^{t+1} = \overline{S/p} \cdot ES^t + S/p \cdot A_1 = D_1^t$$

$$Q_i^{t+1} = \overline{S/p} \cdot Q_{i-1}^t + S/p \cdot A_i = D_i^t$$

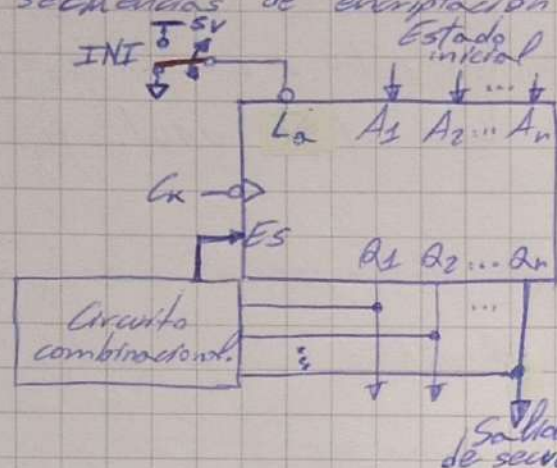
($i \geq 1$)

MXOR de 2 canales



Aplicaciones. Generadores de secuencias

Es un circuito encargado de generar un patrón binario periódico -secuencia binaria de " L " bits- destinada a algún fin concreto (difusión de tramas "piloto" para sincronización y control en redes de paquetes por radio, generación de "testigos" en redes "token ring" y "token bus", secuencias de encriptación para cifradores serie, etc).



La idea consiste en realimentar, por medio de un circuito combinacional, la entrada serie del registro, ES , de forma que este vaya describiendo un grafo cíclico, donde el bit más significativo de cada estado - Q_n - coincidirá con un bit de la secuencia a generar.

Gen. de secuencia $\equiv$ Contador $M = "L"$

Será preciso inicializar previamente el registro con el primer estado del grafo que debe generar.

Procedimiento a seguir para el diseño:

- 1) El número de estados del grafo debe describir el registro, coincidiendo con el número de bits de la secuencia a generar (L), esto nos permitirá determinar el tamaño del registro.

$$\#q'_s = \# \text{ bits secuencia a generar } (L)$$

$$\#q'_s = L \leq 2^n, \text{ donde } "n" \text{ es el n}^\circ \text{ de bits del registro}$$

- 2) Se inicializa el registro con los primeros bits de la secuencia a generar. Este detalle es muy importante, ya que el mismo diseño se podrá emplear para generar tantas secuencias distintas como bits (L) tenga la secuencia original para la que sea diseñado, todo dependerá del estado con que se inicialice el

registro. Por ejemplo, si la secuencia a generar tiene 6 bits, el circuito diseñado para generar dicha secuencia, podría utilizarse para generar otras 5 secuencias más de 6 bits.

El circuito diseñado para generar $\rightarrow \dots / \underline{110} \underline{100} / \dots (q_0 = 6)$

puede generar adicionalmente $\rightarrow \dots / \underline{101} \underline{001} / \dots (q_0 = 5)$

$\rightarrow \dots / \underline{010} \underline{011} / \dots (q_0 = 2)$

$\rightarrow \dots / \underline{100} \underline{110} / \dots (q_0 = 4)$

$\rightarrow \dots / \underline{001} \underline{101} / \dots (q_0 = 1)$

$\rightarrow \dots / \underline{011} \underline{010} / \dots (q_0 = 3)$

- 3) Efectuando desplazamientos para cada estado por el que vaya pasando el registro, determinamos cual deberá ser el bit a realimentar por la entrada serie (próximo estado al que pasar el registro). Este proceso se repite hasta llevar al registro de nuevo a su estado inicial, q_0 .

En caso de aparecer inconsistencias en la tabla obtenida (o gráfico de estados), aumentaremos el tamaño del registro en un bit y repetiremos el proceso de nuevo desde el paso 2.

- 4) A partir de la tabla obtenida diseñamos el circuito combinacional con el que alimentaremos la entrada serie del registro.

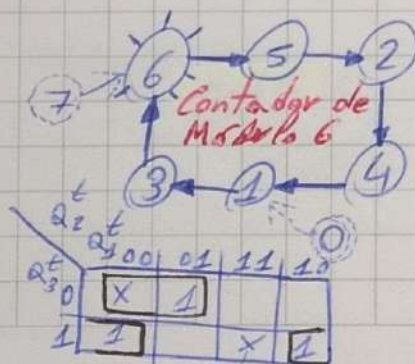
Ejemplo 1: diseñar un circuito que genere la secuencia $\dots / 110100 / \dots$

$\#q_i = \#bits \text{ secuencia} = 6 \text{ estados} \rightarrow$ Registro de 3 bits

Método 1

Q_3^t	Q_2^t	Q_1^t	Est^t
1	1	0	1
1	0	1	0
0	1	0	0
1	0	0	1
0	0	1	1
0	1	1	0
1	1	0	1

$\dots / \underline{110} \underline{100} / \dots$



"Estados indeterminados"

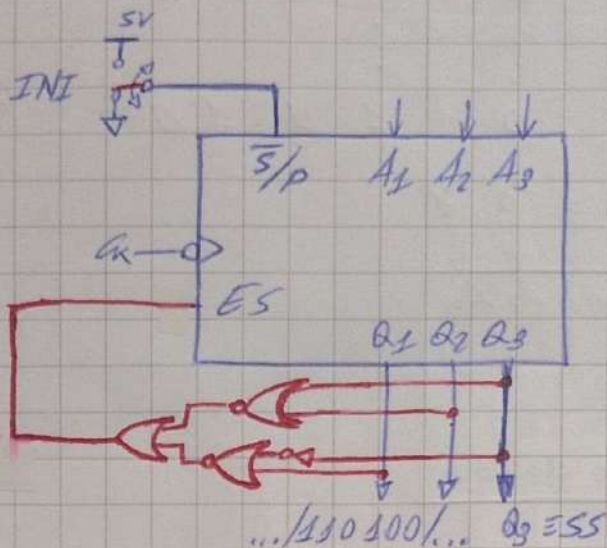
Q_3^t	Q_2^t	Q_1^t	Est^t	Q_3^{t+1}	Q_2^{t+1}	Q_1^{t+1}
0	0	0	1	0	0	1
1	1	1	0	1	1	0

$$\begin{aligned}
 Est^t &= \sum (1, 4, 6) + \sum (0, 7) = \\
 &= \overline{Q_3^t} \cdot \overline{Q_2^t} + Q_3^t \cdot \overline{Q_1^t} = \\
 &= \overline{Q_3^t} + Q_2^t + \overline{Q_3^t} + Q_1^t
 \end{aligned}$$

Método 2

.../110100/...

Q_3^t	Q_2^t	Q_1^t	ES^t
1	1	0	1
1	0	1	0
0	1	0	0
1	0	0	1
0	0	1	1
0	1	1	0
1	1	0	1



Ejemplo 2: diseñar un circuito que genere la secuencia .../10101/...

q_s = # bits secuencia = 5 estados $\rightarrow$ Registro de 3 bits

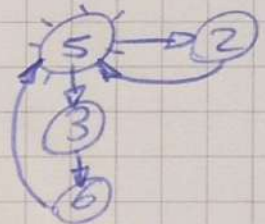
.../10101/...

Q_3^t	Q_2^t	Q_1^t	ES^t
1	0	1	0
0	1	0	1
1	0	1	1
0	1	1	0
1	1	0	1
1	0	1	0

Inconsistencia 1: un circuito combinatorial no puede dar dos salidas diferentes para la misma combinación de entrada.

Inconsistencia 2: no es un contador de módulo 5 (grafo en anillo de 5 estados)

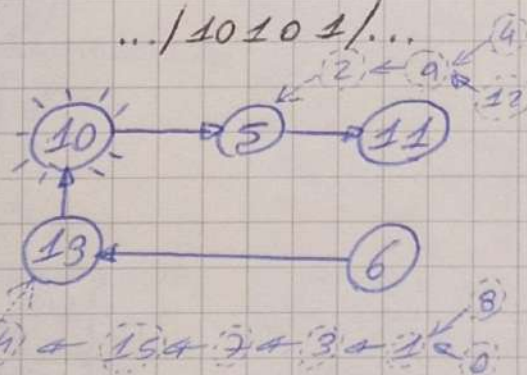
iii No puede generar nuestra secuencia!!!



Aumentamos el tamaño del registro 1 bit y repetimos:

.../10101/...

Q_4^t	Q_3^t	Q_2^t	Q_1^t	ES^t
1	0	1	0	1
0	1	0	1	1
1	0	1	1	0
0	1	1	0	1
1	1	0	1	0
1	0	1	0	1

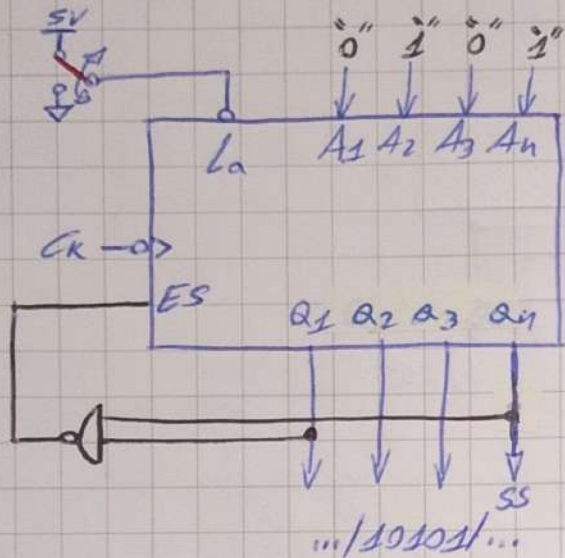


$Q_4^t \backslash Q_3^t$	00	01	11	10
00	X_0	X_1	X_4	X_3
01	X_4	15	X_2	1
11	X_{12}	13	X_{15}	X_{14}
10	X_8	X_9	X_{11}	1

$$ES^t = \sum_3 (5, 6, 10) + \sum_x (0, 1, 2, 3, 4, 7, 8, 9, 12, 14, 15) = \overline{Q_4^t} + \overline{Q_3^t} = \overline{Q_4^t} \cdot \overline{Q_3^t}$$

Estudio Estados indeterminados

Q_n^t	Q_3^t	Q_2^t	Q_1^t	ES^t	Q_n^{t+1}	Q_3^{t+1}	Q_2^{t+1}	Q_1^{t+1}
0	0	0	0	1	0	0	0	1
0	0	0	1	1	0	0	1	1
			$\vdots$	$\vdots$				
1	1	1	1	0	1	1	1	0



R-S NOR
Tabla de transiciones simp.

R^t	S^t	Q^{t+1}
0	0	Q^t
0	1	1
1	0	0
1	1	N.P.

R-S NAND

R^t	S^t	Q^{t+1}
0	0	N.P.
0	1	0
1	0	1
1	1	Q^t

J-K
Tabla de transiciones simp.

J^t	K^t	Q^{t+1}
0	0	Q^t
0	1	0
1	0	1
1	1	$\overline{Q^t}$

Tabla de excitación

$Q^t \rightarrow Q^{t+1}$	R^t S^t
0 $\rightarrow$ 0	x 0
0 $\rightarrow$ 1	0 1
1 $\rightarrow$ 0	1 0
1 $\rightarrow$ 1	0 x

$$Q^{t+1} = \overline{R^t} \cdot Q^t + S^t$$

condición $RS=0$

$Q^t \rightarrow Q^{t+1}$	R^t S^t
0 $\rightarrow$ 0	x 1
0 $\rightarrow$ 1	1 0
1 $\rightarrow$ 0	0 1
1 $\rightarrow$ 1	1 x

$$Q^{t+1} = \overline{S} + RQ^t$$

Tabla de excitación

$Q^t \rightarrow Q^{t+1}$	J^t K^t
0 $\rightarrow$ 0	0 x
0 $\rightarrow$ 1	1 x
1 $\rightarrow$ 0	x 1
1 $\rightarrow$ 1	x 0

$$Q^{t+1} = J^t \cdot \overline{Q^t} + \overline{K^t} \cdot Q^t$$

T (Trigger)
Tabla de transiciones simp.

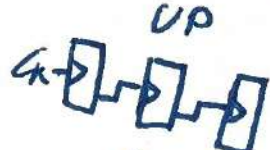
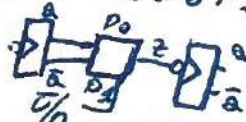
T^t	Q^{t+1}
0	Q^t
1	$\overline{Q^t}$

Tabla de excitación

$Q^t \rightarrow Q^{t+1}$	T^t
0 $\rightarrow$ 0	0
0 $\rightarrow$ 1	1
1 $\rightarrow$ 0	1
1 $\rightarrow$ 1	0

$$Q^{t+1} = T^t \oplus Q^t$$

$$Z_{Mxor} = \overline{S_0} \cdot D_0 + S_0 \cdot D_1$$



Preset y clear

P	C	Q
0	0	N.P.
0	1	1
1	0	0
1	1	Q^t

D (Delay)
Tabla de transiciones simp.

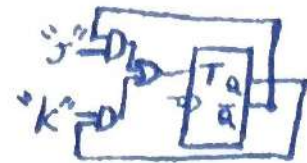
D^t	Q^{t+1}
0	0
1	1

Tabla de excitación

$Q^t \rightarrow Q^{t+1}$	D^t
0 $\rightarrow$ 0	0
0 $\rightarrow$ 1	1
1 $\rightarrow$ 0	0
1 $\rightarrow$ 1	1

$$Q^{t+1} = D^t$$

$T \Rightarrow J-K$



$$T \oplus Q^t = J \cdot \bar{Q}^t + K \cdot Q^t$$

$$T = K \cdot Q^t + J \cdot \bar{Q}^t$$

Método comparación de ecuaciones

$J-K \Rightarrow T$



T	Q^t	Q^{t+1}	J	K
0	0	0	0	x
0	1	1	x	0
1	0	1	1	x
1	1	0	x	1

(Para hallar J y K hacer Karnaugh en función de T y Q^t).

$S-R \Rightarrow D$



D	Q^t	Q^{t+1}	S	R
0	0	0	x	0
0	1	0	1	0
1	0	1	0	1
1	1	1	0	x

Load

L_a	A_i	P_i	C_i
0	0	1	0
0	1	0	1
1	x	1	1

$$P_i = \bar{L}_a \cdot A_i$$

$$C_i = \bar{L}_a \cdot \bar{A}_i$$

J	K	Q^t	Q^{t+1}	T
0	0	0	0	0
0	0	1	1	0
0	1	0	0	0
0	1	1	0	1
1	0	0	1	1
1	0	1	1	0
1	1	0	1	1
1	1	1	0	1

Método tablas de transición y excitación.
(hacer Karnaugh en función de J y K y Q^t).

Salidas: GS (grupo de validación), Entrada Enable (Strobe) = E
 EO (dice las entradas activas).

MUX de 4x2 canales $\rightarrow Z = \bar{S} \cdot D_0 + S \cdot D_1$
 con entrada enable a alta $\rightarrow Z = E(\bar{S} \cdot D_0 + S \cdot D_1)$
74LS157

MUX de 2x4 canales $\rightarrow Z = \bar{S}_1 \cdot \bar{S}_0 \cdot C_0 + \bar{S}_1 \cdot S_0 \cdot C_1 + S_1 \cdot \bar{S}_0 \cdot C_2 + S_1 \cdot S_0 \cdot C_3$
74LS153

Mux de 8 canales $\rightarrow Z = \bar{S}_2 \cdot \bar{S}_1 \cdot \bar{S}_0 \cdot C_0 + \bar{S}_2 \cdot \bar{S}_1 \cdot S_0 \cdot C_1 + \bar{S}_2 \cdot S_1 \cdot \bar{S}_0 \cdot C_2 + \bar{S}_2 \cdot S_1 \cdot S_0 \cdot C_3 +$
 $+ S_2 \cdot \bar{S}_1 \cdot \bar{S}_0 \cdot C_4 + S_2 \cdot \bar{S}_1 \cdot S_0 \cdot C_5 + S_2 \cdot S_1 \cdot \bar{S}_0 \cdot C_6 + S_2 \cdot S_1 \cdot S_0 \cdot C_7$
74LS151

Codificador 8 a 3
74LS148

Decodificador 3 a 8
74LS138

Decodificador 2x2 a 4
74LS139

Comparador 4 bits
74LS85

Cuadruple ST
74LS83

- $\Rightarrow$ NAND 74LS00
- $\Rightarrow$ DAND 74LS08
- $\Rightarrow$ OR 74LS32
- $\Rightarrow$ XOR 74LS86

2x Biestables D (Pr, Cl)
74LS74

2x Biestables JK (Pr, Cl)
74LS76

6 Biestables tipo D (Cl)
74LS174

1/2 + 1/5 Contador
74LS90

1/2 + 1/8 Contador
74LS93

Contador M16 (Pr y M/m)
74LS191

Registro Universal
 4 bits
74LS194

Problema

E	A ₃	A ₂	A ₁	A ₀	S ₂	S ₀
1	X	X	X	X	0	0 (a)
0	0	0	0	1	0	0 (b)
0	0	0	0	0	0	0 (c)

Solución

E	A ₃	A ₂	A ₁	A ₀	S ₂	S ₀	GS	EO
1	X	X	X	X	0	0	1	1 (a)
0	0	0	0	1	0	0	0	1 (b)
0	0	0	0	0	0	0	1	0 (c)

- (a) Si GS y EO = 11 el codificador no está validado
- (b) Si GS y EO = 01 codificador validado y A₀ activa.
- (c) Si GS y EO = 10 codificador validado y ninguna entrada activada.